

ARISTOTLE UNIVERSITY OF THESSALONIKI
FACULTY OF ENGINEERING
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING



Accelerating top-k selection on heterogeneous CPU, GPU and NPU systems

DIPLOMA THESIS

Ioannis Michalainas

ioannisam@ece.auth.gr

Supervisor: Nikolaos Pitsianis

Associate Professor, School of Electrical and Computer Engineering

Thessaloniki, September 2026

Abstract

Note on this translation

Warning! This document was translated from Greek into English automatically with the help of a large language model. The original Greek text is the authoritative version of this thesis. Some passages in this translation may read awkwardly or imprecisely as a result of the automated process.

Top-k selection, the task of identifying the k largest elements of a set, appears across a wide range of applications and is repeated often enough that its cost is noticeable. The recent integration of neural processing units into the same die as the processor makes the question of whether offloading such computations to an accelerator is worthwhile a reasonable one.

This work implements two algorithms with fundamentally different behaviour: truncated bitonic sorting, which is data-oblivious, and map-reduce selection with local heaps, which is data-dependent. The implementations targeted three architectures: a processor with AVX-512 vector extensions, a GPU programmed in CUDA and an AMD XDNA neural processing unit. They were evaluated with a unified methodology across two measurement scopes, algorithmic and end-to-end, recording execution time, energy footprint and the volume of data moved. Their resulting performance was analysed against the system's maximum theoretical capabilities.

The results show that the problem is limited by data movement on every architecture. The ranking of the two algorithms is overturned by the ratio n/k and the choice of measurement scope completely overturns the ranking of the architectures. In energy terms the neural processing unit exhibits the lowest power draw and simultaneously the highest consumption, since duration dominates the product. No accelerator run is Pareto-optimal: for small k , offloading is measurably worse than computing on the processor.

Keywords: top-k selection, bitonic sorting networks, heterogeneous computing, neural processing unit, MLIR-AIE, SIMD vectorisation, energy assessment, roofline model

Acknowledgements

This thesis is the largest programming project I have undertaken to date. I would like to thank my supervisor, Prof. Nikolaos Pitsianis, in whose courses I discovered my interest in high performance computing and low-level programming.

I would also like to thank my family for their unwavering support throughout my studies, especially my brother Manolis, who, as a fellow engineer, helped me with valuable advice and guidance.

Ioannis Michalainas

Contents

Abstract	2
Acknowledgements	3
1 Introduction	9
1.1 Scope of the work	9
1.2 Motivation	9
1.3 Contribution	10
1.4 Structure of the work	10
2 Background and related work	12
2.1 The top-k selection problem	12
2.2 Selection algorithms	13
2.2.1 Bitonic sorting networks	13
2.2.2 Truncated bitonic sort	14
2.2.3 Selection with heaps and map-reduce	15
2.2.4 Radix selection	16
2.3 Processing architectures	17
2.3.1 CPU vector extensions	17
2.3.2 The SIMT model and GPU memory hierarchy	17
2.3.3 The AIE dataflow architecture	18
2.4 Existing implementations and libraries	18
3 Methodology and experimental setup	20
3.1 Reference system	20
3.2 Structure of the measurement harness	21
3.3 Measured quantities	22
3.3.1 Measurement scopes: algo and end-to-end	22
3.3.2 Timing	23
3.3.3 Energy measurement	24
3.3.4 Data traffic measurement	25
3.4 Roofline model	26
3.5 Baseline libraries	27
3.6 Correctness checking	28
3.7 Parameter space	28
3.7.1 Data types	29
3.7.2 Input distributions	29
3.8 Measurement conditions and reproducibility	30

4	CPU implementation	32
4.1	Design choices	32
4.1.1	Vectorisation	32
4.1.2	Vector type abstraction	33
4.1.3	Half-precision floating-point support	34
4.1.4	Parallelisation across multiple threads	34
4.2	Bitonic sort	35
4.3	Map-reduce with local heaps	37
4.4	Memory management and data traffic	38
4.5	Reference implementation	39
4.6	Limiting factors	39
5	GPU implementation	41
5.1	Design choices	41
5.1.1	Mapping onto the grid and thread blocks	41
5.2	Bitonic sort	42
5.2.1	Organisation into tiles and shared memory	42
5.3	Map-reduce with local heaps	43
5.3.1	Parallelism limit for large k	44
5.4	Memory management and data traffic	45
5.5	Reference implementation	46
5.5.1	Radix selection in CUB	46
5.6	Limiting factors	47
6	NPU implementation	48
6.1	Design choices	48
6.1.1	Development flow with MLIR-AIE	48
6.1.2	Tile topology and object_fifo	49
6.2	Bitonic sort	49
6.2.1	Compute tile kernel	50
6.2.2	Scaling to multiple tiles	50
6.3	Map-reduce with local heaps	51
6.4	Memory management and data traffic	51
6.4.1	Pipelining and the role of the host	51
6.4.2	Data encoding and types	52
6.4.3	Final correction on the host	53
6.5	Reference implementation	53
6.6	Limiting factors	53
7	Results and evaluation	55
7.1	Correctness	55
7.2	Execution time	56
7.2.1	Dependence on input size	56
7.2.2	Dependence on the parameter k	57
7.3	Comparison across architectures	60
7.4	Roofline analysis	62
7.4.1	Memory bandwidth utilisation	64
7.5	Energy assessment	65
7.6	Effect of data type	67

7.7 Sensitivity to the input distribution	69
8 Conclusions and future work	71
8.1 Conclusions	71
8.2 Limitations	72
8.3 Future work	72
A Acronyms	74
B Reproducing the Measurements	75
B.1 Software Requirements	75
B.2 Compilation	76
B.3 Running the Experiments	76
B.3.1 Running Individual Steps	77
B.3.2 Execution Parameters	77
B.4 Generating the Plots	78

List of Figures

2.1	Selection versus sorting	12
2.2	The full bitonic network and its truncation	15
2.3	Heap-based selection and the map-reduce scheme	16
2.4	Radix selection	17
3.1	The four levels of the measurement harness	22
3.2	The two measurement scopes per architecture	23
3.3	Reading the roofline model	26
3.4	The five input distributions	30
4.1	The fundamental compare-exchange motif	33
4.2	The half-precision path	34
4.3	Fusing consecutive layers into one pass	36
4.4	Bulk rejection in map-reduce	37
5.1	Sorted block and shared-memory tile	43
5.2	The parallelism limit of map-reduce	45
6.1	Tile topology and object_fifo queues	49
6.2	The three-stage pipelining of the offload	52
7.1	Time as a function of size, under the two measurement scopes	56
7.2	Each algorithm across the three architectures	57
7.3	Time as a function of size, by value of k	58
7.4	The three algorithms on the graphics unit for k equal to 131072.	59
7.5	Speedup relative to the reference implementation	60
7.6	Placement of the runs against the measured ceilings	62
7.7	Measured read bandwidth as a function of working-set size, for the three architectures.	63
7.8	Achieved bandwidth by architecture	64
7.9	Algorithmic and end-to-end time on the graphics unit	65
7.10	Net energy and power by architecture and algorithm	66
7.11	End-to-end time and energy per measured case	66
7.12	Sensitivity of time to the input distribution	69

List of Tables

3.1	The reference system on which all measurements were run.	20
3.2	The content of the two measurement scopes per architecture.	22
3.3	The data types that are swept.	29
3.4	The input distributions that are swept.	29
3.5	The conditions to which the system is pinned before every sweep. . . .	31
6.1	Time breakdown of the offload region by stage	52
7.1	Algorithmic time for the two extremes of k	58
7.2	Algorithmic time by data type	67
B.1	The execution parameters and their defaults.	78

Chapter 1

Introduction

1.1 Scope of the work

This work examines the acceleration of top-k selection, that is, finding the k largest elements of a set of n values, on three architectures: a general-purpose processor, a graphics unit, and a neural processing unit integrated into the same die as the processor.

For each architecture, two algorithms with fundamentally different behaviour are implemented. Truncated bitonic sort executes a fixed sequence of comparisons, independent of the data; selection with local heaps under the map-reduce scheme adapts its work to the input values. The six implementations are measured over the same parameter space, with a common harness and common measurement semantics, so that whatever differences emerge are owed to the algorithms and the hardware and not to the way of measuring.

The central question is under what conditions offloading to an accelerator pays off, weighing both the speed of the architecture and the additional cost of the process.

1.2 Motivation

Neural processing units are now integrated on the same die as the processor of modern laptops, share its main memory, and are available to any application that could make use of them. Whether it is worth offloading this computation to the accelerator has become a timely question for developing software on such hardware.

The answer is not obvious. These units were designed for a specific workload: their software is organised around neural-network operators, and their architecture rests on the assumption that every piece of data transferred will be used many times. What happens when the workload does not have this shape remains largely unexplored.

Top-k selection is a suitable vehicle for posing this question: a problem simple to state, widespread in nearest-neighbour search, retrieval and recommendation systems, the sampling of language models, and many other applications. Yet it

sits at the opposite end of the spectrum from the workloads these accelerators were designed for: it performs minimal operations per element and reuses nothing.

1.3 Contribution

Implementations. Six implementations were built, two algorithms across three architectures, with support for five data types. The processor implementation exploits AVX-512 vector extensions and multiple threads; the graphics unit implementation is organised into tiles with shared memory; the neural processing unit implementation was written entirely at the level of the MLIR-AIE framework. The correctness of every combination was verified over 2700 cases.

Measurement harness. A harness was developed that measures each case in two explicitly distinct scopes, algorithmic and end-to-end. It records energy with common semantics across the three architectures, measures the volume of data moved, and places each run against a roofline ceiling measured on the same machine. It is not built specifically for the top-k problem: it applies to any workload being considered for offloading.

Findings. Top-k selection is a data-traffic problem on every architecture examined; no implementation is limited by computational capability. The ranking of the two algorithms depends on the ratio n/k and stays the same across all architectures, by a margin reaching three orders of magnitude. The measurement scope completely overturns the ranking of the architectures. Energy-wise, the neural processing unit draws the lowest power of all implementations, and yet consumes the most energy, simply because it needs far more time.

Reproducibility. The entire experimental procedure runs from scripts included in the repository. Measurement conditions are recorded alongside the results. Details are given in appendix B.

1.4 Structure of the work

Chapter 2 defines the problem, presents the algorithm families that solve it, and describes the execution models of the three architectures, concluding with the gap this work is called to fill.

Chapter 3 presents what is measured, with what semantics, against which baselines, and under what conditions, a prerequisite for reading the results of Chapter 7.

Chapters 4, 5, and 6 present the implementations on each of the three architectures. They follow a common structure, so that corresponding sections can be compared directly: design choices, the two algorithms, memory management, the reference implementation, and finally the limiting factors identified.

Chapter 7 presents and interprets the results: execution time, comparison across architectures, memory footprint, and energy consumption, across different parameters, data types, and input distributions.

Chapter 8 summarises what generalises, states the limits within which it holds, and proposes directions for continuation.

The appendices include a list of acronyms, a glossary of the Greek terms with their English equivalents, and instructions for reproducing the measurements.

Chapter 2

Background and related work

2.1 The top-k selection problem

Given an array A of n elements from a totally ordered set and an integer $k \leq n$, the top-k selection problem asks for the subset of the k largest elements of A . This work considers the form that asks only for this set, not the order of the elements within it: this is the form required by applications, and the only one against which algorithms with different internal organisation are fairly compared. Every implementation in this work additionally sorts its k results before returning them, which costs $O(k \log k)$ – negligible next to the $O(n)$ of selection itself – and yields a single, checkable answer rather than an arbitrary arrangement of the set.

The critical observation is that the problem asks for strictly less information than sorting: full sorting determines the relative order of all n elements, whereas selection is satisfied with separating them into two groups. This is captured cleanly by the lower bounds – every comparison-based algorithm requires $\Omega(n \log n)$ comparisons to sort, but only $\Theta(n)$ to select with fixed k [1]. Selection is substantially cheaper, and any implementation that sorts the entire input to keep k elements does more work than the problem requires. Figure 2.1 illustrates this difference.

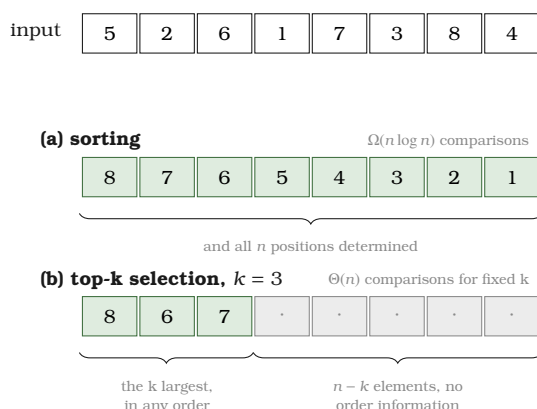


Figure 2.1: The same input under the two requirements. Sorting (a) determines all n positions, while selection (b) splits the elements into only two groups: the k largest, in any order, and the rest, for which no order information is kept.

This gap depends on the ratio k/n . When k is much smaller than n , selection is nearly linear and sorting wastes almost all of its work; as k approaches n , the two problems nearly coincide and the margin disappears. The entire work moves between these two extremes, and chapter 7 shows that the ranking of the algorithms is overturned as this gap is traversed.

The problem appears across a multitude of applications. K-nearest-neighbour search requires selecting the smallest distances from a distance matrix [2]. Information retrieval and recommendation systems return the k best results of a query. In language models, top- k sampling and beam search perform selection at every generation step. In all of these cases, selection is not the primary workload but is repeated often, and its cost becomes noticeable precisely because it is added at every iteration.

2.2 Selection algorithms

Selection algorithms fall into families based on a criterion decisive for acceleration: whether the sequence of operations depends on the data values or not. An algorithm whose schedule is fixed executes the same operations in the same order regardless of the input; a data-dependent algorithm adapts its course, performing less work on favourable inputs and more on adverse ones.

This distinction corresponds to the established classification of parallel selection algorithms into sorting, partial sorting, and partitioning methods [3], but it is not only a matter of theoretical complexity: it also determines which hardware can execute the algorithm efficiently, since parallel architectures presuppose that many processing elements execute the same instruction, and whether execution time is predictable.

2.2.1 Bitonic sorting networks

A sorting network is a fixed sequence of comparators. Each comparator takes two array positions, compares their contents, and swaps them if they are in the wrong order. The positions it compares are fixed in advance and do not depend on the values – the standard example of a data-oblivious algorithm. The same sequence of comparisons is always executed; only the outcome of each comparison changes.

The comparators are organised into *layers*. Within a layer, no position appears in two comparators, so all comparators of a layer execute simultaneously. The layers, however, execute sequentially, so their number constitutes the depth of the network and defines the lower bound on time on unboundedly parallel hardware.

Batcher’s construction rests on the notion of a bitonic sequence [4]: one that first increases and then decreases, or results from such a sequence by a cyclic shift. Its fundamental property is that a bitonic sequence of length $2m$ sorts in $\log_2(2m)$ layers: comparing each element with the one m positions away yields two bitonic sequences of length m , where every element of the first is less than or equal to every element of the second. The two halves are then sorted independently by the same procedure.

Full sorting of n elements results from repeated application: pairs of sorted blocks of opposite direction form a bitonic sequence of double the length, which is sorted to yield a block of double the size. The network has $\log_2 n (\log_2 n + 1)/2$ layers and $n/2$ comparators per layer, i.e., $O(n \log^2 n)$ comparators in total – a factor of $\log n$ more than the $O(n \log n)$ bound achieved by sequential sorting algorithms, so bitonic sort performs more comparisons than necessary. The trade-off is that these comparisons are entirely predetermined and uniform, executable on vector units, SIMT models, and dataflow arrays without any control-flow divergence. More operations for perfect regularity: this trade-off is why sorting networks remain competitive on parallel hardware despite their asymptotic disadvantage.

2.2.2 Truncated bitonic sort

Full sorting produces far more information than selection requires. Truncated bitonic sort exploits this by discarding elements as soon as it is proven that they cannot belong to the result [2]. The same idea, halving the active array at every step, also underlies partial-sorting implementations for graphics units [5].

Figure 2.2 contrasts the two structures. Let k' be the smallest power of two not less than k . The procedure has two phases. In the first, ordinary bitonic sort is executed but stops as soon as sorted blocks of size k' are formed, instead of continuing to the single block of size n . In the second, blocks are merged in pairs, and of the $2k'$ elements in each pair, only the k' largest are kept, with the rest discarded permanently. This step halves the size of the active array and repeats until a single block remains, containing the desired result.

The correctness of this discarding is immediate: if an element is not among the k' largest of two sorted blocks, then k' elements larger than it already exist in the array, so it cannot be among the k largest of the whole set.

The gain is twofold. The first phase requires $\log_2 k' (\log_2 k' + 1)/2$ layers instead of $\log_2 n (\log_2 n + 1)/2$ – a count depending on k , not on n . The layers of the second phase act on an array that shrinks geometrically, so their cost sums to a term linear in n . The overall complexity falls from $O(n \log^2 n)$ to $O(n \log^2 k)$.

For $n = 2^{24}$ and k equal to 8, the full network has 300 layers and 2 516 582 400 comparators, while the truncated network has 90 layers and 92 274 668 comparators: twenty-seven times fewer. For k equal to 131072, however, the figures are 279 layers and 1 441 595 392 comparators, only 1.7 times fewer than the full network. The benefit of truncation evaporates as k grows.

The total number of comparators in the truncated network is given by

$$\frac{n}{2} \cdot \frac{\log_2 k' (\log_2 k' + 1)}{2} + \frac{(\log_2 k' + 2)(n - k')}{2},$$

where the first term corresponds to the first phase and the second to the merge-and-truncate step.

Truncation does not sacrifice data-obliviousness, though: which elements are discarded depends on the values, but *how many* and at which steps does not, since the schedule of layers is computed in advance from n and k alone.

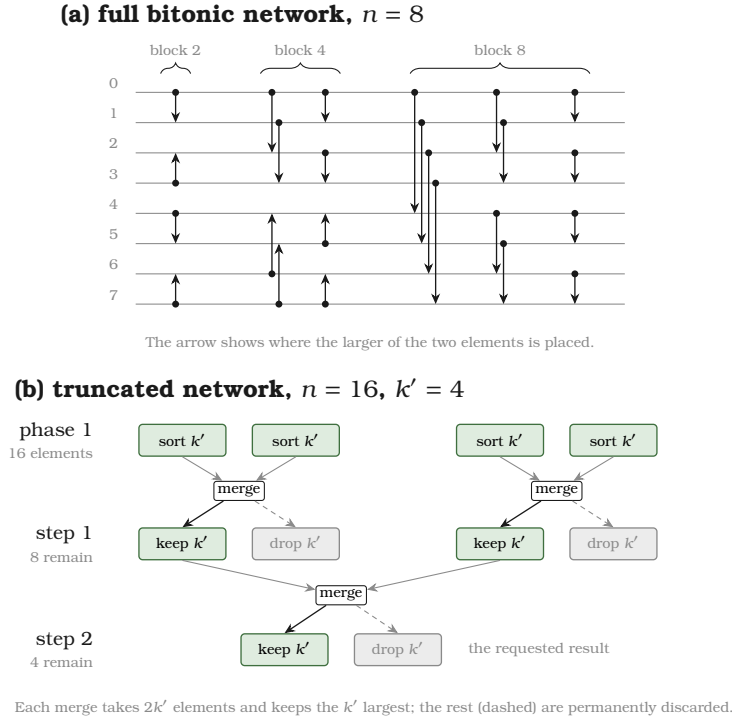


Figure 2.2: The full bitonic network and its truncation. In (a) each horizontal line is an array position and each vertical arrow a compare-exchange. The network is data-independent, since it always executes the same sequence of comparisons in the same order, regardless of the values. In (b), when only the k' best elements are requested, the first phase stops as soon as sorted blocks of size k' have formed, and in the second phase each pairwise merge keeps only the k' best of the $2k'$ compared. The array size is halved at each step, so subsequent steps cost progressively less.

2.2.3 Selection with heaps and map-reduce

The second family follows the opposite logic: instead of imposing a fixed schedule, it maintains a structure holding the k best elements examined so far and updates it only when necessary.

The structure is a min-heap of capacity k . Its root holds the smallest of the k current best elements and acts as an admission threshold: for each new element, a single comparison against the root is performed, and if the element does not exceed it, it is discarded with no further work. Otherwise it replaces the root, and the heap is restored in $O(\log k)$ steps.

This asymmetry determines the algorithm's behaviour. On random input, the threshold rises quickly and the overwhelming majority of elements are discarded with a single comparison, so the total time approaches linear. On ascending input the opposite happens: every new element exceeds the root and forces a restoration, giving $O(n \log k)$ in the worst case.

Parallelisation follows the map-reduce scheme, shown in figure 2.3: in the map phase the input is partitioned and each worker maintains its own local heap without synchronising with the others; in the reduce phase the partial results are merged into one. The scheme is attractive because the map phase requires no communica-

tion. It imposes a limit that proves critical, though: each local heap must first fill with k elements before it can start filtering, so the number of workers that actually contribute cannot exceed the ratio n/k . This limit is negligible on a processor with eight threads, but disastrous on a graphics unit with thousands, as analysed in section 5.3.1.

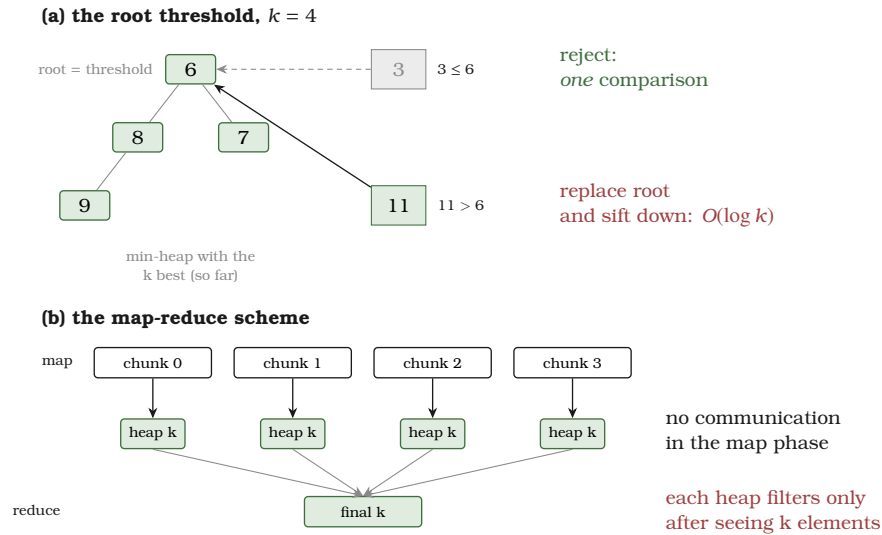


Figure 2.3: Heap-based selection and its parallelisation. In (a) the root of the min-heap acts as a threshold: an element that does not exceed it is rejected with one comparison, while one that exceeds it replaces the root and triggers a sift-down. In (b) the input is partitioned into chunks, one heap per worker, and the partial results are merged into one.

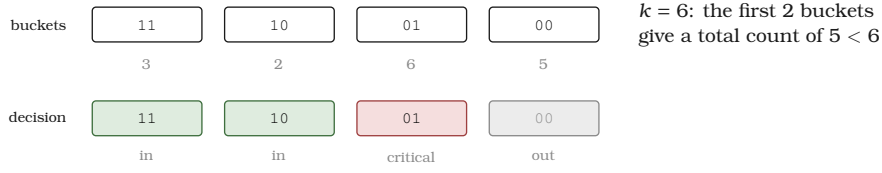
2.2.4 Radix selection

The third family does not compare elements against each other but examines their binary representation. Values are distributed into buckets according to a group of bits, the bucket counts are summed, and from these it follows which bucket contains the k -th largest element: buckets above it belong entirely to the result, buckets below it are discarded entirely, and the search continues only within the critical bucket, using the next group of bits.

The procedure is shown in figure 2.4. Two properties stand out: the number of passes is determined by the width of the data type and not by k , so the method exhibits no collapse point as k grows; and each pass is a sequential read with frequency counting, a pattern particularly favourable for high-bandwidth architectures.

This method is also data-dependent, but differently from heaps: what matters is not the order in which values appear, but the distribution of their bits. When many elements share the same significant bits, the critical bucket narrows slowly and more passes are required. This case is known in the literature as a radix-adversarial distribution and is addressed with adaptive candidate-storage strategies [3]. This family was not implemented in the present work, but it underlies the graphics unit's reference library (section 2.4).

pass 1: the 2 most significant digits



pass 2



Figure 2.4: Radix selection. At each pass the elements are distributed into buckets according to a group of digits and the count of each is tallied; from the cumulative counts it follows which buckets belong entirely to the result, which are rejected, and which is the *critical* one. No comparison between values is ever performed.

2.3 Processing architectures

The three architectures examined in this work expose parallelism in fundamentally different ways.

2.3.1 CPU vector extensions

The modern general-purpose processor combines two forms of parallelism: multiple threads, each executing an independent instruction stream, and vector extensions, which apply the same operation to multiple data items simultaneously within a single thread.

An AVX-512 register is 512 bits wide and holds sixteen 32-bit integers. Beyond arithmetic operations, its repertoire includes shuffle instructions that permute elements within the register and min/max instructions that select element-wise without branching. The combination of shuffle, min, and max implements exactly one sorting-network comparator, which makes bitonic sort a natural candidate for vectorisation.

The limiting factor is usually not the rate of operations but their supply of data. The cache hierarchy partly compensates for the speed gap with main memory, but only for algorithms that reuse data while it remains at a nearby level.

2.3.2 The SIMT model and GPU memory hierarchy

The graphics unit is organised around the SIMT model. Threads are grouped into warps that execute the same instruction on different data. When the threads of a warp follow different branches, the branches execute serially with the inactive threads disabled: control-flow divergence therefore has a direct cost, and branch-free algorithms have an advantage.

Warps are grouped into thread blocks, and blocks into a grid. Each block has shared memory, a small region managed explicitly by software, with latency far lower than global memory. This is the tool by which a multi-pass algorithm is converted into a

single-pass one: if the chunk of data processed by a block fits there, all intermediate steps execute without returning to global memory.

Access to global memory is efficient only when the threads of a warp request consecutive addresses, so the individual requests are coalesced into few transactions. Finally, the graphics unit is a discrete device: data must be transferred to and from its memory over a bus whose bandwidth is an order of magnitude lower than that of local memory.

2.3.3 The AIE dataflow architecture

The term neural processing unit has meant different things over time, from early proposals for approximate, energy-saving computation [6, 7] to the dedicated neural-network accelerators [8] whose performance-per-watt advantage was established once deployed at data-centre scale [9].

Central to understanding these units is the design principle stated explicitly in the Eyeriss architecture: moving a piece of data costs orders of magnitude more energy than the operation performed on it, so design must be organised around maximising local data reuse [10]. In convolution and matrix multiplication this principle is fully vindicated, since every element loaded participates in many operations. Top-k selection lies at the exact opposite extreme: each element is examined a minimal number of times and there is essentially no reuse to exploit.

The architecture examined in this work belongs to AMD’s AI Engine family, introduced with the Versal adaptive systems [11] and subsequently integrated, as XDNA, into general-purpose processors [12].

The execution model is spatial, fundamentally different from the previous two. The unit consists of an array of tiles, each with its own vector processor and its own local memory. There is neither shared memory nor cache memory, so communication happens exclusively through explicit data transfer over an interconnect network.

The consequence is that the program is not expressed as a sequence of operations on a shared address space, but as a graph of compute kernels connected by streaming channels: the programmer specifies which tiles execute which kernel and how data flows between them. The abstraction this work uses for this purpose is the `object_fifo` of the MLIR-AIE framework [13], presented in section 6.1.2.

The local memory of each tile is small and fixed, so the computation must be broken into chunks that fit within it, with the partial results merged elsewhere. Feeding the array is also the host’s responsibility, which must prepare and forward every chunk. When the computation per element is small, as in selection, the cost of feeding dominates over the cost of computation.

2.4 Existing implementations and libraries

The availability of ready-made implementations differs dramatically across the three architectures, and this disparity shapes both the question posed by this work and the choice of baselines in section 3.5.

For the processor, the C++ standard library offers `std::nth_element`, which partitions the array around the k -th element in expected linear time, and `std::partial_sort`, which additionally sorts the result by maintaining a heap of size k . Both are universally available and well-tested, but strictly single-threaded: neither exploits multiple threads nor the hardware’s vector extensions.

For the graphics unit, CUB, part of the CCCL library collection [14], provides DeviceTopK, a radix selection implementation as described in section 2.2.4. It is a mature and highly optimised solution, used here as a baseline without attempting to surpass it across the entire parameter range.

The extent to which algorithm choice depends on k is recognised in industry too: recent work on AMD graphics units identifies exactly the same overturn recorded in section 7.2.2 and addresses it with an adaptive strategy – register-level bitonic sort for small k and radix selection for large k , with an empirically determined transition point [15].

Alongside exact methods, a distinct line of *approximate* algorithms has also developed, trading some recall for speed and targeting mainly machine-learning workloads, where top- k selection proves to be a systematic bottleneck on accelerators designed for dense matrix multiplication [16]. The present work is restricted to exact selection.

For the neural processing unit no equivalent library exists: its software ecosystem is oriented towards neural-network operators and does not include a top- k selection primitive. The closest available material is an official bitonic sort example on a single AI Engine tile [17], which, however, performs *full* sorting, only for single-precision numbers, and for sizes up to 1024 elements that fit in one tile’s local memory. This work instead addresses selection rather than sorting, five data types, and inputs of up to 2^{24} elements requiring pipelining across multiple tiles: this is precisely the gap the present work is called to fill.

The absence of a ready-made solution has two consequences: every implementation must be built from scratch at the level of MLIR-AIE [13], which makes this unit the most demanding part of the work, and there is no natural on-device baseline, so this work uses the host implementation as the reference, a choice justified in section 6.5.

Overall, the literature on top- k selection for graphics units is extensive and covers both sorting-network-based and radix-based methods [2, 15]. The corresponding literature for AIE-type dataflow accelerators, by contrast, is limited, since these units are recent and their use focuses on convolution and matrix-multiplication workloads.

Chapter 3

Methodology and experimental setup

3.1 Reference system

All measurements in this work come from a single system, which carries all three architectures being compared: a general-purpose processor (CPU), a discrete graphics unit (GPU), and a neural processing unit (NPU) integrated into the same die as the processor. This isolates implementation differences from platform differences, but introduces a constraint: the system is a laptop, so the three units share a common thermal and power budget, an issue handled as described in section 3.8.

System	Lenovo Legion 5 15AHP10
Operating system	Arch Linux x86_64, kernel 7.1.5-arch1-2
Main memory	32 GiB DDR5-5600, dual channel
CPU	AMD Ryzen 7 260
Cores / threads	8 / 16, up to 5.10 GHz
Cache	8 MiB L2, 16 MiB L3
Vector extensions	AVX-512
GPU	NVIDIA GeForce RTX 5060 Laptop
Memory	8 GiB
Software	driver 610.43.03, CUDA 13.3
Power limits	50 W default, 115 W maximum
NPU	AMD XDNA (RyzenAI-npu1)
Software	XRT 2.21.75, firmware 1.5.5.391

Table 3.1: The reference system on which all measurements were run.

The processor has eight physical cores with simultaneous multithreading, and all measurements run with one thread per physical core, so that no two threads share the same vector unit. The cache hierarchy sets the knee points of the measured bandwidth scale (section 3.4): depending on input size and data type, the working set may fit in L2, in L3, or only in main memory, and each implementation is compared against the ceiling that matches its own working set.

Main memory is arranged as two identical 16 GiB modules in dual-channel mode, at 5600 MT/s. A single-channel arrangement would halve the available bandwidth, and a memory-bound implementation would give substantially different results on an otherwise identical system.

The neural processing unit has no memory of its own; it is fed from the same DDR5, and through the same memory controllers, as the processor, on the same silicon die. Two consequences of this recur throughout this work: the cost of preparing data on the host is not separable from the cost of the computation, and the energy meter used covers the whole package rather than the AIE tiles individually (section 3.3.3).

The entries of table 3.1 were verified from the system itself, not from manufacturer datasheets.

3.2 Structure of the measurement harness

The measurement harness is organised into four layers around one principle: the measurement procedure itself is common to all architectures, and only the compute kernel differs. No implementation, therefore, controls how it is itself measured.

The first layer is a shared C++ core that all three implementations share. It reads the parameters, generates the input, schedules the bitonic network's layers, reads the energy meters, runs the warm-up and measurement loop, checks correctness, and prints the report. All of this happens in one place, so that the timing and energy measurements of every architecture are defined by the same code.

The second layer consists of the per-architecture executables. Each implements the same interface for the three compared algorithms – bitonic sort, map-reduce with local heaps, and the reference implementation – providing only the function that performs the computation, which the shared measurement loop of the first layer calls.

The third layer is the sweep driver: it enumerates the parameter space of section 3.7 and runs each case as a separate process, recording its output verbatim together with its outcome, so that each case starts from a clean memory state and the failure of one does not invalidate the sweep.

The fourth, finally, converts the raw results into tables and plots, on top of the same report text the user reads. There is no separate output path to the tools, so the values in the plots are by construction the values printed by the executable. The execution environment is stored alongside the results, as described in section 3.8, and the commands that produce each stage are given in appendix B. The overall structure is summarised in figure 3.1.

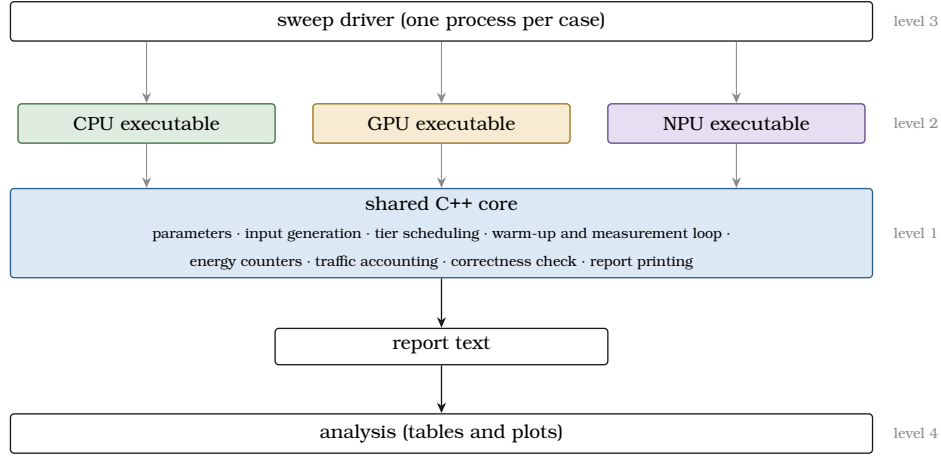


Figure 3.1: The four levels of the measurement harness. The measurement procedure is defined only once, in the shared core of the first level, and the three implementations provide exclusively the function that performs the computation.

3.3 Measured quantities

3.3.1 Measurement scopes: algo and end-to-end

Comparing three architectures presupposes a decision about what exactly is counted as cost. An accelerator that finishes the computation very quickly, but requires the whole input to be transferred to and from main memory, is not necessarily faster than a processor working in place. This is why every case is measured in two scopes, not one.

The end-to-end scope covers the entire call from the host’s side: everything required so that, given an input in main memory, the result is also found in main memory. The algo scope covers only the region in which the computation itself runs. Their difference is the cost imposed by heterogeneity itself, summarised per architecture in table 3.2 and schematically in figure 3.2.

Architecture	Content of algo	Difference from end-to-end
CPU	identical to end-to-end	—
GPU	region between CUDA events	H2D/D2H transfers, memory allocations
NPU	region of the offload	device discovery, loading the offload configuration

Table 3.2: The content of the two measurement scopes per architecture.

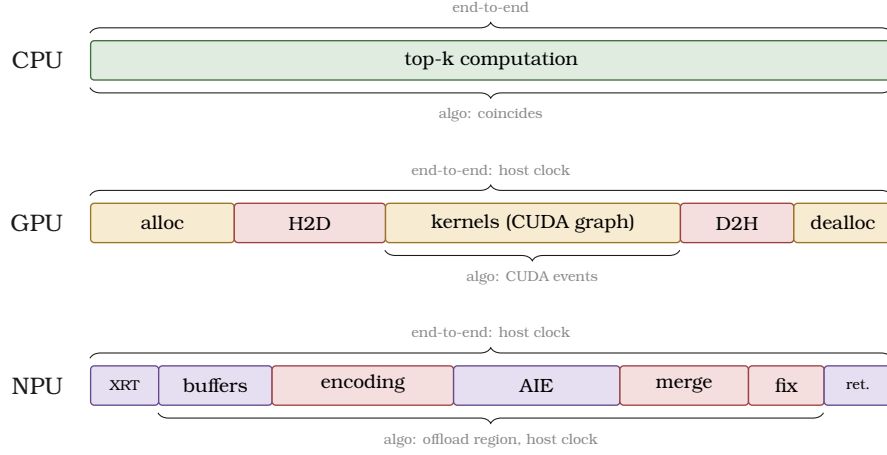


Figure 3.2: The two measurement scopes per architecture. The color of each stage indicates which unit its work belongs to, with red for the extra cost imposed by each architecture: bus transfers for the graphics unit, encoding, merging, and final correction on the host for the neural processing unit.

On the processor the two scopes coincide by definition: it is itself the accelerator, and there is no host cost to subtract. Any other choice would grant it an advantage it does not deserve.

The energy meter opens and closes at exactly the same points in the code as the corresponding timer, so time and energy refer to the same region of execution and are directly comparable. A third, wider window around the entire measurement loop is additionally recorded as an upper-bound sanity check; it is discussed in section 3.3.3.

The choice of scope determines the question a measurement answers. End-to-end is the fair scope for comparison across architectures, since none of them hides its own data-transfer cost. Algo is useful for comparison within the same architecture, but does not suffice for claims across architectures, precisely because it omits the transfers.

3.3.2 Timing

The two scopes are not necessarily measured with the same mechanism. The end-to-end scope is always measured with a host clock, on either side of the call, since by definition it concerns time as perceived by the host. The algo scope is measured with whatever mechanism each architecture provides for the region of its own computation.

On the graphics unit the measurement cannot be done with a host clock. Kernels launch asynchronously: the call returns as soon as the kernel is placed in the command stream, not when it finishes. A host clock would therefore measure the time of the launch rather than of the execution, while forced synchronisation would add its own cost to the kernel. Events, recorded within the command stream and timed by the device itself, are used instead.

On the neural processing unit there is no equivalent device-side meter; the algo

scope is therefore defined by a host clock around the region of the offload. This region does not contain only the execution on the tiles: it includes allocating and preparing the buffers, dispatching the jobs, and the final merge of the partial results on the host. It must therefore be read as the cost of the entire offload procedure, not as the compute time of the AIE tiles – a distinction essential to the conclusions of chapter 6.

From the 50 measured repetitions of each case, three quantities follow per scope: the mean, on which the comparisons are based; the standard deviation, an indication of the measurement’s stability under the pinned conditions of section 3.8; and the minimum, a lower bound on the achievable time. The meters have nanosecond-scale resolution, against measured regions ranging from a few tens of microseconds to hundreds of milliseconds, so the cost of the timing itself is negligible and is not subtracted.

3.3.3 Energy measurement

Energy is measured by the executable itself, not by an external wrapper that monitors the process, for a specific reason: an external tool measures the whole process, whereas the measurement loop occupies only part of it, a part that moreover differs systematically by architecture, which would introduce bias in favour of architectures with lower initialisation cost. The meters are therefore read from within the process, at exactly the same points in the code as the timers of section 3.3.2.

The sources differ by architecture, but the scope remains symmetric. For all architectures the energy of the processor package is measured via the RAPL counters, while for the graphics unit the card’s power draw is added. The end-to-end scope thus corresponds to the total cost of solving the problem for all three, without the graphics unit concealing the cost it imposes on the host.

The reported energy is the sum, not the mean, of the 50 repetitions. The RAPL counters quantise each individual reading, but accumulate without reset, so the sum stays accurate even when each individual difference is on the order of the quantisation step. Reading the counters requires elevated privileges, and when these are missing the executable states explicitly that the measurement is unavailable, rather than reporting zero values as data.

Of the RAPL fields, only package is used: the core subfield was found unreliable on this processor, recording only 6 W under full eight-thread load, while package power rises by about 53 W.

For the graphics unit, the total-energy counter of the NVML library also proved unsuitable. It advances in steps of about 20 J every few tens of milliseconds, so bracketing a millisecond-scale kernel returns an arbitrary multiple of the step: three identical runs gave values differing by more than a factor of four, while the corresponding time differed by 0.2 %. Instead, a helper thread samples the card’s instantaneous power every 2 ms and integrates it over time. The power reading itself refreshes less often than this sampling period, so an algo window of a few milliseconds relies on a reading that has not had time to refresh. Summing over the 50 repetitions smooths out this inaccuracy without eliminating it, unlike the RAPL counters, which refresh on a millisecond scale.

Beyond the gross value, net energy is also reported, i.e., the gross value minus the energy the system would consume idle over the same interval, with idle power measured before every sweep for the full set of counters of each architecture. The net figure is preferable especially for the neural processing unit, where the static consumption of the rest of the system dominates, and for the graphics unit, where it cancels out the cost of the sampling mechanism itself.

This methodology has an inherent limitation, worth stating explicitly. The package meter covers the entire die: host cores, NPU tiles, uncore, and memory controller, all under one shared meter. A run on the neural processing unit is therefore measured as the total energy of the system while it works, including host-side data preparation. This is a fair, symmetric comparison of the cost of solving the problem, but not proof that the unit performs the computation while itself consuming less energy, since its own consumption is never separated from the host's; isolating it would require external metering equipment.

3.3.4 Data traffic measurement

Beyond time and energy, every run records the bytes it moved and the operations it performed – the coordinates that place the run on the plot of section 3.4. An error in counting them does not merely introduce imprecision; it can lead to an entirely wrong conclusion about what limits the implementation.

For this reason traffic is not estimated from a model, but accumulated by the code itself at the point where it occurs: per group of fused passes on the processor, per kernel launch on the graphics unit, per data batch on the neural processing unit. The obvious estimate, element count times type size, assumes that every element is read once – an assumption that does not hold here. Truncated bitonic sort traverses the working set repeatedly, by a factor that is not constant: roughly sevenfold the naive estimate for k much smaller than n , rising considerably and differing by architecture for larger k , where truncation stops paying off. The naive estimate would make every implementation appear to use only a small fraction of the available bandwidth, wrongly suggesting that none of them approach the memory ceiling.

The count is checked by independent verification wherever feasible. For bitonic sort with k up to 4096, the processor and the graphics unit run the same layer schedule and compute their traffic in entirely separate code, yet arrive at the same value down to the last byte – strong evidence that the model is correct. For map-reduce the two values differ, as expected: although both maintain a local heap per thread, the processor runs a handful of threads while the graphics unit runs thousands, with a correspondingly larger working space and an additional tree merge within each thread group. The two implementations genuinely move different volumes of data.

Not every quantity is equally observable, and every run states the quality of its own numbers. For bitonic sort the bytes and the operation count are exact, since the schedule is fully enumerable. For map-reduce both the bytes and the comparison count are lower bounds, because the merge traffic and the reordering of the local heaps are data-dependent. For the reference implementation both quantities are

lower bounds, since the libraries' internal passes are not observable from the calling code.

Operations are measured in compare-exchanges rather than floating-point operations: top-k selection performs no arithmetic operations, so a figure in FLOPs would not correspond to anything actually executed, whereas the compare-exchange is the natural unit of work for the problem.

3.4 Roofline model

Execution time says how fast an implementation is, not why it is not faster. The roofline model [18] answers this, comparing every measurement against the machine's two upper bounds: a bandwidth β and a compute capability π . Which of the two binds is a property neither of the machine alone nor of the algorithm alone, but of their relationship, expressed by the operational intensity, the ratio of operations W to bytes Q moved:

$$I = \frac{W}{Q}, \quad P = \min(\pi, \beta I)$$

Figure 3.3 shows how the plot is read.

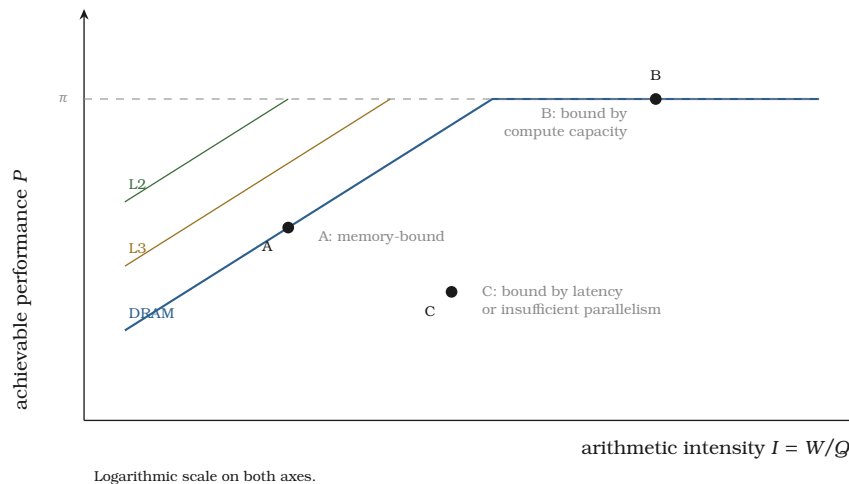


Figure 3.3: Reading the roofline model. The sloped segment corresponds to the bound βI imposed by memory bandwidth, and the horizontal one to the bound π imposed by compute capacity; the achievable performance is the minimum of the two. Because memory bandwidth depends on the size of the working set, the sloped segment is not a single line but a ladder of parallel lines, one per level of the memory hierarchy, and each measurement is compared against the one that corresponds to its own working set. Point A is memory-bound and will benefit only from a reduction in data traffic, B from compute capacity, while C is far from both ceilings: there the limiting factor is not the hardware but the organisation of the execution.

Applying the model here departs from usual practice in three respects, necessary for the comparisons to be honest.

First, both ceilings are measured on the same machine with separate executables, rather than taken from manufacturer datasheets, since what matters is what is

actually achieved, not what the hardware could achieve under ideal conditions.

Second, the memory ceiling is not just one number: achievable bandwidth depends on the size of the working set, since data arrive far faster while they fit in some level of cache than from main memory. Bandwidth is therefore measured over successive working-set sizes, starting at 32 KiB and doubling each time, and every measurement is compared against the ceiling of its own working set. This scale measures bandwidth under reuse, which might seem unsuitable for an algorithm that traverses the input only once; the objection does not hold here, though, since the input is restored from its backup copy immediately before the timed region (section 3.8), so that for working sets up to the size of the last cache level the data are already present in it and the measurement corresponds to a second pass. Beyond that size the scale converges on its own to the rate of main memory.

Third, the compute ceiling is measured in compare-exchanges, for the reason given in section 3.3.4. If every compare-exchange used the result of the previous one, a dependency chain would form, measuring the latency of a single operation instead of the throughput of executing many, and the ceiling would come out several times lower, with every implementation wrongly appearing to be touching it. The measurement kernel therefore runs 64 independent chains in parallel, alternating between three registers so that no operation is elided by the compiler.

For the neural processing unit the model carries a different meaning: there is no available AIE kernel with tunable operational intensity, so the measured ceilings concern the unit’s feed path rather than the computation on the tiles. They are read as “how fast the host can prepare the data the unit consumes” – the implementation’s actual bottleneck.

3.5 Baseline libraries

The performance baseline is an established library implementation per architecture. A new implementation has value only if it is faster than what the programmer already has available.

For the processor, `std::partial_sort` is used. Its choice over `std::nth_element` is not self-evident: for k much smaller than n it is clearly faster, since it maintains a heap of size k that stays in cache and rejects most elements with a single comparison.

For the graphics unit, `cub::DeviceTopK` is used, which implements radix selection. It matters that this is a genuine top-k selection function and not a full sort followed by truncation, so the comparison is against an opponent that solves the same problem in an optimised way.

For the neural processing unit there is no equivalent baseline. The AIE toolchain provides a compiler and example kernels, not an algorithms library: no established top-k implementation exists for this hardware, neither from the manufacturer nor from third parties. The same `std::partial_sort` used for the processor is used instead, run entirely on the host without any use of the unit. This changes the question the comparison answers: not “is our implementation better than an optimised AIE kernel”, since no such kernel exists, but “is it worth offloading the computation to

the unit, or is it preferable to run it with the means the host already has” – exactly the question anyone actually considering the unit’s use faces in practice.

The shared baseline additionally offers an independent check on the harness: because it is literally the same code on the same hardware, the two measurements ought to agree, and their agreement in the results of chapter 7 confirms that the harness introduces no systematic bias depending on which architecture is declared.

3.6 Correctness checking

A timing measurement is meaningful only if the result is correct, so every case of every sweep is verified, and those that fail are recorded as failures rather than included among the results.

Checking is done against a separate ground truth, which must not be confused with the baseline libraries of section 3.5: those are a performance baseline, while this one judges whether a result is correct. It is the simplest possible implementation: a copy of the input is fully sorted on the host and the first k elements are kept. It is slow, with no intention of competing; its only role is to be obviously correct, so that its verdict can be trusted.

The comparison is not made the same way for every type: integers require exact equality, floating-point types a tolerance, relative or absolute depending on the magnitude of the values, since different architectures perform operations in a different order and with different intermediate precision. Because every implementation returns its result sorted, in the same order as the ground truth (section 2.1), the comparison is made position by position rather than as a set. When both the full and the truncated network are run, the two results are additionally cross-checked against each other, as an independent confirmation that truncation does not corrupt the result.

In this work’s main sweep all 2700 cases were verified with no failures.

3.7 Parameter space

Every run is fully determined by its command-line parameters. None is read from a configuration file or environment variable, so that the description of a measurement is always complete and reproducible. The full list is given in appendix B.

The input size is a power of two, n equal to 2^q , with q swept from 3 to 24. The upper bound, about 16.8 million elements, is large enough that the working set exceeds the cache by far and the measurement concerns main memory. The lower bound is not arbitrary either, since for q smaller than 3 the input has fewer elements than the smallest requested k .

The parameter k takes the values 8, 256, 4096, and 131072, orders of magnitude apart, covering situations from a k so small that the heap fits in a handful of registers, to a k that is a significant fraction of the input itself. Where k exceeds n , it is clamped to n .

3.7.1 Data types

Five data types are swept, given in table 3.3.

Type	C++ mapping	Size	Description
int	std::int32_t	32 bit	signed integer
uint	std::uint32_t	32 bit	unsigned integer
float	float	32 bit	single-precision floating point
double	double	64 bit	double-precision floating point
half	_Float16	16 bit	half-precision floating point

Table 3.3: The data types that are swept.

The type affects the measurement in two ways: it determines how many bytes move per element, so the position of the same implementation relative to the memory ceiling shifts with the type, and it determines the cost of a comparison, which is not the same across types.

3.7.2 Input distributions

The distribution of the input values is not immaterial: heap-based algorithms do work only when a new element exceeds the heap's root, so the order in which values appear drastically changes their running time. The five distributions of table 3.4 are swept. The executable additionally supports the normal distribution, not used here since trimodal covers the case of clustered values more sharply. The shape of each is shown in figure 3.4.

Distribution	Description
uniform	uniform over the value range
trimodal	three narrow clusters within the range
sorted	ascending order
reverse	descending order
adversarial	values in the lower half with ascending spikes

Table 3.4: The input distributions that are swept.

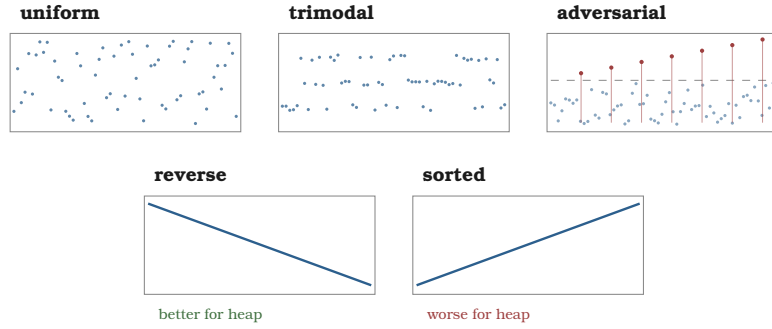


Figure 3.4: The five input distributions, with the value of each element as a function of its position in the array. The uniform and trimodal distributions have no ordering and differ only in which values occur; the two ordered ones are the extremes for heap-based selection. In adversarial the bulk of the values stays below the midpoint of the range (dashed), with isolated spikes at regular intervals.

The two ordered distributions form the two extremes for heap-based selection (section 2.2.3): descending is the most favourable case and ascending the least favourable. Adversarial retains this unfavourable behaviour without being fully sorted: most of the values lie in the lower half of the range, with ascending spikes at regular intervals.

Bitonic sort is data-oblivious, since it executes the same comparison network regardless of the values, and so serves as a witness: if its time changes across distributions, the change is due to the measurement and not to the algorithm.

3.8 Measurement conditions and reproducibility

The system is not, by default, in a state suitable for measurement. In its default configuration the frequency governor is powersave and frequency boost is active, with the result that operating frequency varies with load and temperature. This is a systematic drift over the course of a sweep as the system heats up, not random noise, so the standard deviation of samples taken under drifting conditions would not capture the measurement's true uncertainty.

For this reason, before every sweep the system is pinned to a fixed, documented state, summarised in table 3.5, recorded before the sweep and restored afterwards. The corresponding commands are given in appendix B.

Parameter	State during measurement
CPU frequency governor	performance
CPU frequency boost	disabled
CPU threads	8, one per physical core
GPU persistence mode	enabled
GPU power limit	fixed for the whole sweep
Cool-down period between units	15 s
Warm-up repetitions	5 per case
Measured repetitions	50 per case

Table 3.5: The conditions to which the system is pinned before every sweep.

The graphics unit’s power limit is chosen once, within the bounds of table 3.1, and kept fixed. Its value is arbitrary, but any change to it during a sweep would alter time and energy at the same time.

As noted in section 3.1, the three units share a common thermal budget. A 15 s cool-down period between successive units keeps each one from starting out from the thermal state left by the previous one, and no other computational load runs during a sweep. This does not eliminate the problem, only mitigates it: on a laptop the thermal coupling between the units cannot be fully removed, which remains an acknowledged limitation of these measurements.

Every case is run 5 times as warm-up and 50 times under measurement, with the mean, standard deviation, and minimum reported. Before every repetition the input is restored from a backup copy, outside the timed region, so that the cost of the copy is not counted and every repetition starts from an identical input. This is critical for data-dependent implementations: without this care they would show artificially improved times on an already partially sorted input.

Beyond the repetitions within a case, every combination of distribution and random-number-generator seed constitutes an independent sample. The plots aggregate these samples and depict their spread as error bands, distinguishing uncertainty due to the data from uncertainty due to the measurement.

Finally, every run records the environment in which it ran: the repository revision, the kernel, the compiler, and the driver and runtime library versions of each unit. In this way, every set of measurements documents its own production conditions without depending on external records that might drift out of sync.

Chapter 4

CPU implementation

4.1 Design choices

Four decisions run through both implementations in this chapter and are described here once: how vector instructions are exploited, how the same algorithm supports five data types, how half precision is handled, and how work is distributed across threads.

4.1.1 Vectorisation

Both algorithms rely on an operation that vectorises exceptionally well. The compare-exchange of two elements is equivalent to computing their minimum and maximum, and both of these operations exist as instructions that act on an entire vector register at once, without branches. A 512-bit register performs sixteen single-precision integer comparisons in the time of one. The instructions are referred to here by the names of the corresponding intrinsic functions of the standard [19], also implemented by the processor described in section 3.1.

The difficulty lies not in the comparison but in positioning the elements. To compare two elements separated by a distance j , they must be brought to the same position in two different registers. When j is smaller than the register width, this is achieved by shuffling within the register: the contents are shifted by j positions, the minimum and maximum are computed, and the result is reassembled by alternately selecting from the two using a mask. The sequence “shuffle, minimum, maximum, select” is the fundamental motif on which the layer fusion of section 4.2 is built, depicted in figure 4.1.

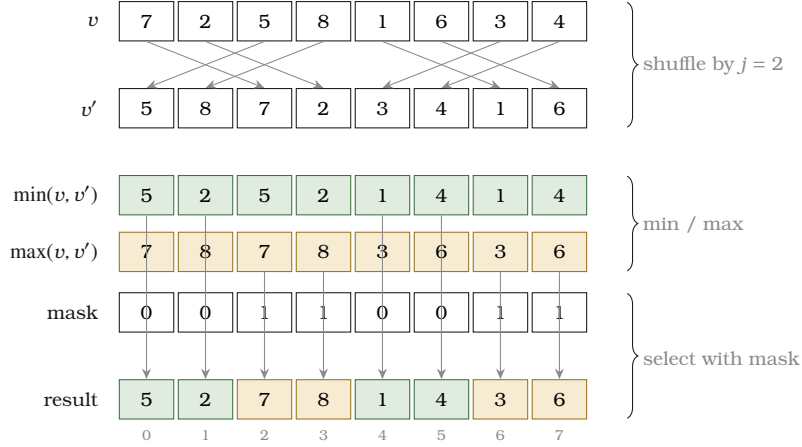


Figure 4.1: The fundamental “shuffle, min, max, select” motif for compare-exchange at distance $j = 2$, in an eight-lane register. The contents are permuted so that each element ends up opposite its counterpart, minimum and maximum are computed simultaneously for all lanes, and the result is reassembled by selecting the minimum where the corresponding mask bit is zero and the maximum where it is one. No branch intervenes and no value is examined individually.

The register width is not known at compile time. The code is compiled with all paths present, and the choice is made at start-up, by querying the processor for the instructions it supports: if AVX-512 is available, the 512-bit registers are selected, otherwise the 256-bit ones, and as a last resort the scalar path. The same executable therefore runs on any machine, with the difference that on different hardware it follows a different code path and its results are not directly comparable.

4.1.2 Vector type abstraction

Vector instructions are specialised by data type: one instruction computes the minimum of signed integers, another of unsigned integers, another of floating-point numbers. With five types, two register widths, and two algorithms, writing the code directly would lead to twenty nearly identical variants of the same code, with a matching maintenance cost and the risk of one drifting from another.

Instead, a single interface is defined. For each combination of type and width a structure is declared that always exposes the same names: the vector type, its width in elements, load and store, minimum, maximum, masked select, shuffle, and the generation of the candidate mask used by map-reduce. The algorithms are written once against this interface and never refer to a specific instruction.

This architectural abstraction secures two key advantages: adding a new data type requires only a new specialisation of the structure, with no changes to the algorithms, and the absence of a specialisation is caught early, at compile time. If no interface is defined for some combination of type and width, the code automatically falls back to the scalar path instead of failing. The equivalence of the paths is checked.

4.1.3 Half-precision floating-point support

Half precision is a special case, because the processor stores 16-bit values but has no arithmetic instructions that operate directly on them at the width the algorithm requires. The obvious solution, scalar processing, would negate any benefit.

The solution adopted exploits the fact that conversion between the two representations is a single instruction. The half-precision specialisation of the interface defines its vector type as that of single precision: the load reads sixteen 16-bit values and converts them to sixteen 32-bit values, all operations execute in single precision, and the store converts them back. The data remain in half precision in memory and are in single precision only for the duration of the computation, as shown in figure 4.2.

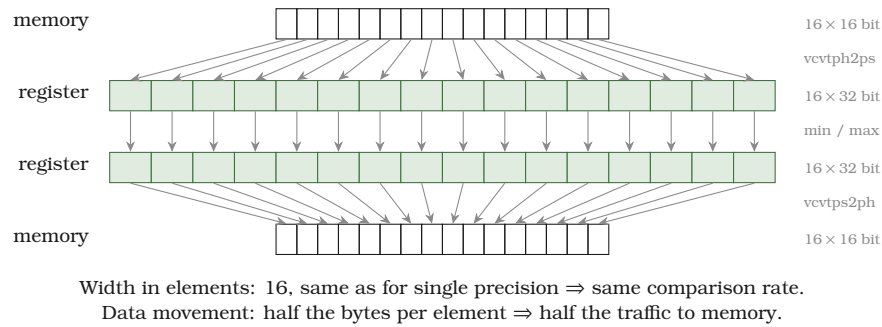


Figure 4.2: The half-precision path. The data remain in 16 bit in memory, where they occupy half the space, and are converted to single precision only within the registers, for the duration of the computation.

This approach is not approximate. Every half-precision value is represented exactly in single precision, and the only operations performed are minimum and maximum, which select one of the two inputs rather than compute a new value. No rounding intervenes: the result is identical to that of a hypothetical implementation that operated natively in half precision.

The register width in elements remains sixteen, the same as for single precision, so the rate of comparisons does not change. Data traffic, however, is halved, since each element occupies half the bytes. For memory-bound algorithms, as both were shown to be, this translates directly into a speed-up.

4.1.4 Parallelisation across multiple threads

The two algorithms are parallelised in fundamentally different ways, and this difference determines how aggressively each can be scaled.

Bitonic sort has global dependencies: each group of layers requires the result of the previous one to be complete. Parallelism is therefore confined to within a group, with a synchronisation barrier between groups. The advantage is that threads maintain no private state, and their cost is limited to synchronisation.

Map-reduce has the opposite structure: threads work entirely independently on disjoint segments and meet only once, at the merge. No synchronisation is required

during the scan, but each thread maintains its own heap of size k , and the final merge is serial and proportional to the number of threads.

The optimal number of threads is thus not the same for the two. Bitonic sort tolerates more threads on smaller inputs, whereas map-reduce requires a substantially larger share of work per thread to amortise the cost of the extra heap and the serial phase. In both cases the thread count is dynamically limited according to input size, so that parallelism never costs more than it returns.

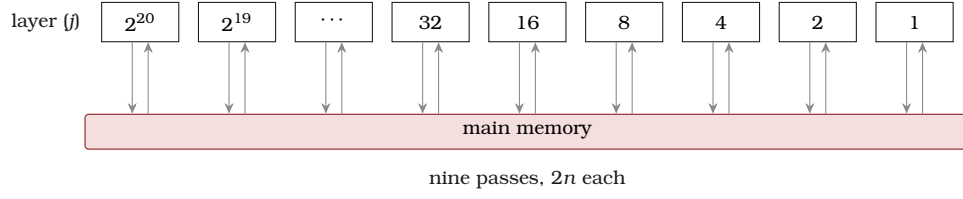
4.2 Bitonic sort

The layer schedule of section 2.2.2 is built once and is shared across all three architectures: a first phase forms sorted blocks of size k' , the smallest power of two not less than k , and a second phase merges them in pairs, keeping the best k' of the $2k'$ compared each time.

A direct translation of this schedule into code produces an implementation that is hopelessly memory-bound. For k equal to 256, the first phase alone consists of 36 layers, each of which would be a separate pass over the entire array. For an input of four million elements this means dozens of consecutive 16 MB reads and writes, with almost no computation in between. This work's entire implementation is organised around avoiding exactly this pattern.

The basic technique is fusing consecutive layers into a single pass. A layer is characterised by the distance j between the elements compared, and this distance determines what is feasible. When j is smaller than the vector register width, the two elements of every comparison already lie within the same register, and a whole sequence of such layers then executes as a single load, a series of in-register shuffles and comparisons, and a single store. When j is larger but the working set of a chunk fits in the cache, the sequence executes chunk by chunk, with each chunk staying in cache for all its layers. Only layers with very large j require a separate pass. In addition, a truncation layer whose distance equals the register width also absorbs the re-sort layers that follow it. The two schemes are contrasted in figure 4.3.

(a) naive transfer



(b) layer fusion

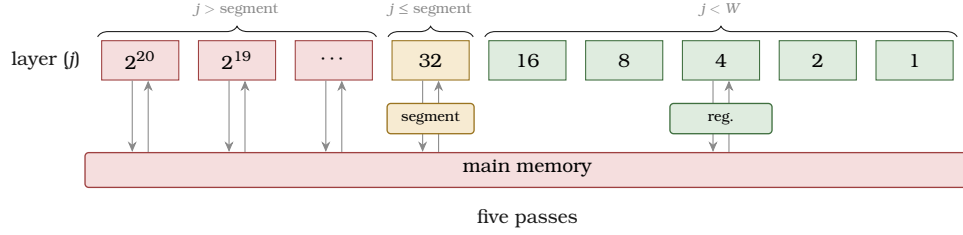


Figure 4.3: The fusion of consecutive layers into a single pass; each pair of arrows is one read and one write of the entire array. In (a) each layer is a separate pass to memory; in (b) the layers are grouped by distance j and only the last group retains a separate pass. The numbers in the figure are illustrative.

The resulting data traffic is predictable from the structure of the schedule. The first phase collapses into a single in-place pass, that is, one read and one write per element, hence $2n$. Each step of the second phase reads n_i elements and writes $n_i/2$, and the re-sort that follows adds a further $2 \cdot n_i/2$, for a total of $2.5 n_i$. Because n_i halves at each step, the sum over the second phase is a geometric series:

$$\sum_i 2.5 n_i = 2.5 \left(n + \frac{n}{2} + \frac{n}{4} + \dots \right) = 2.5 \times 2n = 5n.$$

Total traffic is therefore about $7n$ elements, which is the origin of the repetition factor of section 3.3.4. For $n = 2^{22}$ and k equal to 256, this expression gives 117 435 392 bytes, a value that matches the measured one exactly.

Concurrent execution across multiple threads takes place within each group, not between groups, since each group depends on the result of the previous one. The output range of each group is divided among the threads, with boundaries aligned to the chunk size or the cache line so that two threads never write to the same line. A synchronisation barrier intervenes between groups, and the thread count is further limited according to input size, so that for small inputs the cost of synchronisation does not exceed the benefit of parallelism.

Truncation layers cannot execute in place, because each output element results from two input elements at different positions. For this reason a second array of the same size is maintained, and the two arrays are swapped after each such group – the extra memory is the price paid for avoiding intermediate copies.

Finally, every vectorised path is accompanied by an equivalent scalar one. If the hardware does not support the required width, or if the parameters of a layer fall outside the range handled by the vectorised implementation, the same sequence of

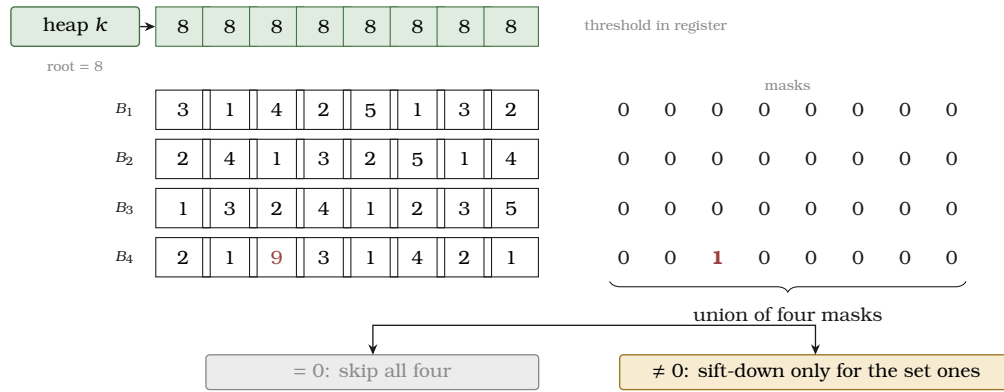
layers is executed in scalar form. The result is identical, and only the time differs, which lets the correctness check validate both paths.

4.3 Map-reduce with local heaps

The second algorithm follows an entirely different logic from the first. Rather than imposing a fixed sequence of comparisons independent of the data, it maintains the k best elements seen so far and tests every new element against them. The input is split into segments, one per thread; each thread produces its own k best, and the partial results are merged at the end.

Each thread maintains a heap of size k , whose root is the worst of the k elements selected so far. The heap is bulk-initialised from the first k elements of the segment, which costs linearly in k and is cheaper than k successive insertions. From then on, the root acts as a threshold: any element that does not exceed it can be rejected with no further work.

The critical point of the implementation is that this rejection happens in bulk. The threshold is broadcast into a vector register and compared simultaneously against an entire block of elements, producing a binary mask that shows which of them are candidates. The implementation examines four such blocks together and, if the union of the four masks is zero, skips all four with a single check. In the common case, where the heap has already filled with large values and most elements are rejected, the bulk of the input is traversed exactly this way: read, compared vectorwise, and rejected, with no element ever examined individually. The process is depicted in figure 4.4.



In the common case, the left path is followed almost always.

Figure 4.4: Bulk rejection in map-reduce. The root of the heap is broadcast across an entire vector register and compared simultaneously with four blocks of the input, producing one binary mask per block. The union of the masks selects one of the two paths at the bottom of the figure.

Only for the bits that are actually set is scalar work performed. For each such element, the root is replaced and the heap restored by sifting down. Replacing the root is preferred over extraction followed by reinsertion, since the latter would traverse the heap twice instead of once.

This is exactly where the data dependence described in section 3.7.2 originates. If the input is descending, the heap fills immediately with the largest values, the threshold rises to its maximum possible value from the outset, and the entire rest of the input is rejected in bulk. If it is ascending, the opposite happens: every new element exceeds the root, no mask is ever zero, and a full sift-down is executed for every element. The same code, given an input of the same size, performs almost no work beyond reading in one case, and the maximum possible in the other.

The merge phase is simple and serial. The elements of each local heap are offered in turn to the first one, using the same threshold criterion, and the final result is sorted. The cost of this phase is proportional to the product of the number of threads and k , and does not depend on input size, so for large n it is negligible next to the scan.

Data traffic follows directly from this structure: every input element is read exactly once, and the merge moves twice the product of the number of threads and k . This figure is therefore exact. The comparison count, by contrast, is reported as a lower bound, precisely because the depth of each sift-down depends on the values; the count records one comparison per element and ignores the reordering of the heap.

The thread count is limited far more conservatively than in bitonic sort: roughly one million elements per thread are required. The reason is that every additional thread adds another heap of size k and burdens the serial merge, so that beyond some point parallelism costs more than it returns. For an input of four million elements, consequently, four threads are used rather than eight, even when eight are requested.

4.4 Memory management and data traffic

The two algorithms solve the same problem on the same machine, but their memory behaviour is radically different. Contrasting them is instructive, because most of the performance difference is explained by this alone.

The fundamental difference is the volume of traffic. From the expressions in sections 4.2 and 4.3, bitonic sort moves approximately $7n$ elements, while map-reduce moves n plus a term proportional to the product of the number of threads and k , negligible for small k . For an input of four million integers, the first moves 117 MB and the second 17 MB. The ratio of seven to one comes from the algorithm itself, not the implementation: the sorting network is obliged to traverse the data many times, while heap-based selection traverses it exactly once.

Equally important is the nature of the access. Map-reduce reads the input and never modifies it, so its traffic is purely sequential reading, the most favourable pattern possible for the hardware prefetcher. Bitonic sort, by contrast, writes on every pass, and the comparisons take place between elements separated by a variable distance, which at some layers exceeds the entire chunk that fits in cache.

The memory footprint also differs. Bitonic sort requires a second array equal in size to the first, so it allocates $2n$ elements in total. Map-reduce allocates only the local heaps, that is, the product of the number of threads and k , independent of input

size. For small k this total is small enough to remain permanently in the fastest levels of the hierarchy, and the only traffic to main memory is reading the input itself.

The two algorithms therefore sit at different points relative to the ceilings of section 3.4. Both perform few operations per byte, so neither approaches the compute ceiling, but map-reduce achieves a substantially higher transfer rate than bitonic sort, as expected from its more favourable access pattern. Bitonic sort pays twice over: it moves a sevenfold volume, and it moves it at a lower rate.

This advantage is not universal, though. The term relating to the local heaps grows with k , and for the larger values examined it ceases to be negligible: the heap no longer fits in the fastest cache levels, and the merge phase, being serial, acquires a weight comparable to the scan. Bitonic sort, whose traffic depends much more weakly on k , incurs no equivalent penalty.

4.5 Reference implementation

The baseline for the processor is `std::partial_sort` from the standard library, for the reasons explained in section 3.5. It is integrated into the same harness as this work's two implementations, runs in the same measurement loop, and is verified by the same correctness check, so its times are directly comparable.

There is, however, an asymmetry that affects how every speed-up is measured. The library function is strictly single-threaded: it accepts no thread count and uses only one core. Bitonic sort, by contrast, uses up to eight threads and map-reduce up to four or eight, depending on input size, so every ratio of times between this work's implementations and the library conflates two distinct causes: the difference in algorithm and the difference in core count.

This choice is not arbitrary. The central question this work asks is the practical speed-up each architecture offers, measured against the standard solution a developer already has available. Since the established solution for the processor is indeed single-threaded, it is used as the realistic baseline. The reader should nonetheless bear in mind that a stricter “thread-for-thread” evaluation would give smaller speed-ups.

4.6 Limiting factors

The limiting factor is not the same across the full range of k . In fact, this shift is decisive enough to change which of the two implementations is faster.

For small k and large input, both implementations are clearly memory-bound. At the largest input examined, bitonic sort approaches four-fifths of the measured bandwidth ceiling and map-reduce three-quarters, while both utilise around five percent of the compute ceiling. The gap between the two figures leaves no room for doubt about the nature of the limitation.

This diagnosis has a practical consequence for any attempt at improvement. Bitonic sort is not slow because its comparisons are inefficient, but because it moves a

sevenfold volume of data. No further optimisation of the compute kernel is going to speed it up substantially; the room for improvement lies exclusively in reducing traffic. Map-reduce, on the other hand, already reads every element exactly once and operates close to the rate at which memory can feed it; under this condition it is essentially optimal, in the sense that it cannot be made faster without reading less.

For large k the picture changes radically, and neither of the two approaches either ceiling. At the largest value of k examined, utilisation of both ceilings falls to figures on the order of one percent for map-reduce and a little above ten percent for bitonic sort. When a measurement is far from both ceilings, the real limiting factors are latency and insufficient parallelism, rather than bandwidth or compute capability.

For map-reduce, as k grows the heap's threshold falls and the rejection rate collapses: the fast path that skips four blocks at once stops activating, and almost every element forces a full sift-down. At the same time the heap stops fitting in the fastest cache levels, so every sift-down becomes a sequence of random-address accesses of depth logarithmic in k . The algorithm turns from bandwidth-bound to latency-bound, and at the largest value of k it ends up slower than bitonic sort.

A secondary but real factor is the thread-scaling limits described in the preceding sections. Because map-reduce requires roughly one million elements per thread, only part of the available cores is used at medium input sizes. Part of the gap from the ceiling in this region is therefore not due to the algorithm but to the processor not working at full capacity.

Chapter 5

GPU implementation

5.1 Design choices

Two issues run through both implementations in this chapter: the handling of data types and the mapping of work onto the device's execution structure.

Type handling follows the same logic as on the processor, but in a simpler form. A structure is defined per type that exposes the operations the algorithms need, namely comparisons and the minimum and maximum, and the algorithms are written once against it. The essential difference from the processor's case is that here there is no question of selecting a width at run time: the compiler generates code for the device's specific architecture, so the abstraction serves only to distinguish types.

A mapping between the host's types and the device's types is also required, since half precision is represented by a different type in the two environments. This mapping happens at a single point and is transparent to the algorithms, which have no knowledge of which side they run on.

5.1.1 Mapping onto the grid and thread blocks

The device executes threads organised into blocks, and blocks into a grid. The distinction is not administrative: threads of the same block can share fast memory and synchronise with one another, while threads of different blocks have neither capability. Each algorithm must therefore place its dependencies within blocks wherever this is possible.

Both implementations use blocks of 256 threads, a size that balances having enough threads to cover memory latency against not so many that the number of blocks able to coexist on each multiprocessor is restricted.

The way the grid size arises differs between the two algorithms. In bitonic sort it is determined strictly by the data: each block takes on one tile, so the number of blocks matches the number of tiles into which the active segment is split, a direct mapping with no freedom of choice. In map-reduce, by contrast, the grid is a matter of design, constrained by the occupancy the hardware allows, the heap-filling condition, and the available workspace (section 5.3.1).

Regardless of grid size, the order in which threads access memory is decisive. Simultaneous accesses by neighbouring threads are coalesced into single transactions only if they touch neighbouring addresses.

5.2 Bitonic sort

The layer schedule is the same as in section 4.2, since it is built from shared code. What changes is how it is executed, while the problem the implementation must solve stays the same: the number of layers is large, and the naive mapping of one layer to one kernel launch would mean dozens of successive passes over global memory.

On the graphics unit a second cost is added that does not exist on the processor. Every kernel launch carries a fixed overhead, and a scan with hundreds of launches pays this overhead hundreds of times. For this reason the entire network is not launched kernel by kernel, but is instead captured once as an execution graph and then replayed; scheduling thus happens once rather than for every layer.

The kernels that compose the graph are of three kinds, and the choice between them is determined by the distance j of the layer, exactly as on the processor. When the distance is small, an entire sequence of layers executes inside the shared memory of a thread block, as described in section 5.2.1. When it is too large to fit in a tile, the data stays in global memory and a single kernel executes up to five successive layers, holding the intermediate results in registers. The truncation layers, finally, have their own kernel, which reads two elements and writes one to a second array, with the two arrays alternating just as on the processor.

5.2.1 Organisation into tiles and shared memory

The shared memory of a thread block is explicitly managed, unlike the processor's cache, which operates automatically. The implementation loads each block's tile into it (section 5.1.1), executes as many layers as possible entirely within it, and writes it back once. Any layers fused this way cost no additional traffic to global memory.

For 32-bit integers the tile ends up at 8192 elements, a size arrived at through testing with larger and smaller tiles, revolving around two constraints: on the one hand it cannot exceed the 48 KB available per thread block without special configuration, and on the other it shrinks when the working set fits in the device's second-level cache, where the bottleneck shifts from traffic toward main memory to the number of blocks simultaneously active.

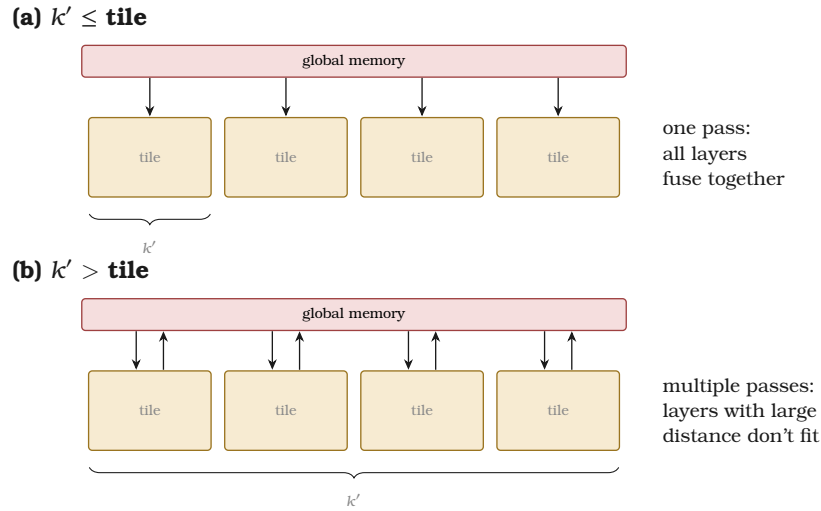
The constraint is not specific to this implementation: published partial-sorting methods for graphics units are subject to the same bound and do not support k beyond a few hundred, precisely because their structures must fit in shared memory and in registers [3, 5].

This limit determines the behaviour of the whole implementation. A layer can only be fused if the distance between the elements it compares fits in the tile, that is, if it is smaller than half the tile's size. Because the distances of the first phase reach up

to half the size of the sorted blocks, the critical size is the k' of section 4.2: as long as the blocks fit in the tile, almost all layers are fused.

Figure 5.1 contrasts the two cases. For small and medium values of k this holds, and the data traffic turns out essentially identical to that of the processor: about seven times the naive estimate, with the two architectures agreeing down to the last byte for k up to 4096, as noted in section 3.3.4, since the same schedule executes with an equivalent fusion strategy, only in explicitly managed memory instead of automatic.

For the largest value of k examined, this condition breaks down. The sorted blocks reach a size of 131072 elements, sixteen times larger than the tile, so most of the first phase's layers can no longer be fused and instead execute as passes over global memory. Measured traffic rises correspondingly, from sevenfold to between twenty-two and twenty-nine times the naive estimate depending on input size.



Measured traffic goes from $7n$ to $22n$ to $29n$ once k' exceeds the tile.

Figure 5.1: The relationship between the sorted block and the shared-memory tile. A layer can execute without returning to global memory only if the distance between the elements it compares fits in the tile. As long as sorted blocks of size k' fit in it (a), nearly all layers fuse into one pass and data traffic matches that of the processor. Once k' exceeds the tile size (b), layers with large distance execute as separate passes over global memory and traffic multiplies. For 32-bit integers the tile holds 8192 elements, so the transition falls between the two largest values of k examined.

5.3 Map-reduce with local heaps

The algorithm is the same as in section 4.3: each thread maintains a heap of size k with the best elements it has seen, the root acts as a threshold, and the partial results are merged at the end. The difference lies in the scale of the decomposition.

On the processor the threads are few and each takes on a contiguous segment of

the input. On the graphics unit the threads are thousands, and stride through the input with a stride equal to their total number, so that simultaneous accesses by neighbouring threads touch neighbouring addresses and are coalesced into single memory transactions.

Each thread maintains its own heap, not one shared per block. The heaps do not fit in shared memory, since the product of the number of threads and k far exceeds the 48 KB available per block, so they are kept in a workspace in global memory. This is exactly where the implementation diverges from the processor's in terms of data traffic: the workspace is proportional to the product of the threads and k , and with thousands of threads this term ceases to be negligible. Because each thread requires a heap of size k , the total number of threads is subject to an upper bound on memory, binding for large k .

The merging happens at two levels. Within each thread block, the heaps are combined pairwise in a tree, with a number of steps logarithmic in the block size. The result of each block is then merged with the rest. This structure avoids the serial phase that burdens the processor, but introduces its own cost, since every step of the tree reads and writes heaps from global memory.

5.3.1 Parallelism limit for large k

The constraint is binding exactly where the graphics unit would be most useful. For the largest value of k examined and an input of sixteen million elements, the ratio n/k is a mere 128. A device with thousands of cores can therefore substantially occupy only one hundred and twenty-eight threads, a fraction on the order of one percent of those available.

The algorithm's effectiveness rests entirely on rejection: an element that does not exceed the heap's root costs only a single comparison. The root, however, only becomes meaningful as a threshold once the heap is full. As long as it holds fewer than k elements, every new element is inserted regardless and no rejection takes place.

This gives rise to the exact constraint. If an input of size n is split among P threads, each thread sees n/P elements and its heap fills only if $n/P \geq k$, that is,

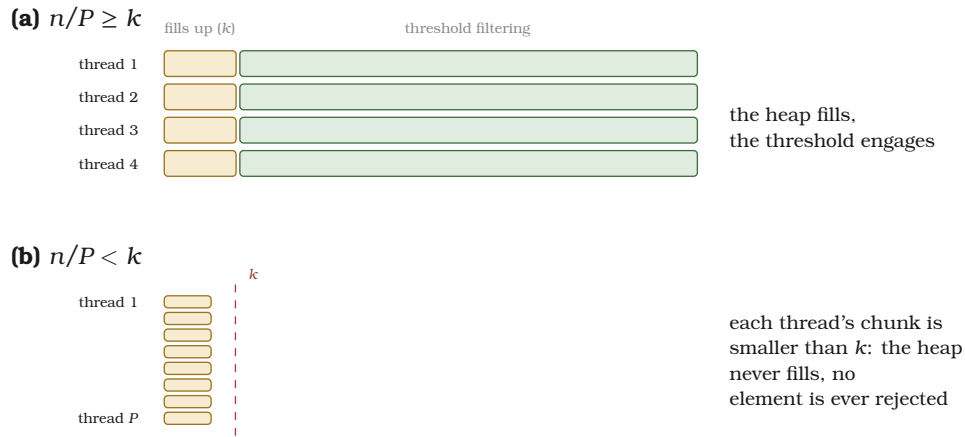
$$P \leq \frac{n}{k}$$

the upper bound on the number of threads that can contribute substantially. Beyond it, the extra threads do not speed up the computation but burden it: they fill the workspace with heaps that never filter and add levels to the merge tree. No code avoids this underutilisation, since it stems from the way the algorithm itself works.

The implementation reacts to this condition. Before launch it computes whether the heaps manage to fill under full occupancy of the device. When this holds, the number of blocks is deliberately restricted, inversely proportional to k , so that each thread takes on a larger share of the input and its heap filters effectively: nominal parallelism is sacrificed for actual rejection. When the heaps are not going to fill

under any circumstance, the reduction would offer nothing, and the device is used at full occupancy (figure 5.2).

In every run, the stricter of the two constraints prevails. The memory limit of section 5.3 acts alongside this one, together explaining why the implementation is unable to make use of the device at large values of k .



Useful parallelism is therefore bounded by $P \leq n/k$, regardless of grid size.

Figure 5.2: The parallelism limit of map-reduce for large k . Each thread maintains its own heap of size k and can reject elements only after the heap fills, i.e., only after it has seen at least k elements. When the chunk assigned to each thread is larger than k (a), the threshold engages early and the rest of the chunk is filtered. But when the number of threads increases enough that the chunk becomes smaller than k (b), no heap ever fills and filtering never engages. This is a property of the algorithm, not the implementation: neither the grid size nor the workspace layout can remove it.

5.4 Memory management and data traffic

The graphics unit is characterised by two separate memory systems connected by a bus incomparably slower than either. The device's local memory delivers about 320 GB/s, while the bus to the host delivers a mere 14 GB/s: a difference greater than twentyfold.

Within the device, the picture resembles that of the processor. For small k , bitonic sort moves the same sevenfold volume. Map-reduce's traffic, though, depends noticeably on input size, due to the per-thread workspace (section 5.3).

What actually dominates, though, is the transfer to and from the device. For an input of four million integers, the roughly seventeen MB take more than one millisecond just to arrive, at a point where the bitonic sort kernel itself completes in half that time: the transfer, that is, costs more than twice the computation it serves.

This work measures two scopes (section 3.3.1); a comparison relying solely on the algorithmic scope would credit the graphics unit with an advantage it does not have once the problem is framed as data transfer to and from main memory.

The established technique for improving transfers, namely the use of pinned host memory to avoid an intermediate copy, offers no substantial benefit here: measurements of the two paths differ by a margin on the order of two percent, which indicates that the limiting factor is the bus itself, not the way data is prepared on either side of it.

As for the memory footprint on the device, the two implementations follow the logic of their processor counterparts: bitonic sort allocates the two arrays of size n from section 5.2, and map-reduce the per-thread heaps from section 5.3. For large values of k , the heap workspace can approach the size of the input itself.

5.5 Reference implementation

The baseline for the graphics unit is the top- k selection function of the CUB library, which ships with the CUDA toolkit. Unlike the processor’s case, where the reference implementation is single-threaded and the comparison conflates two causes, here there is no such asymmetry: the library runs on the same device, exploits the same parallelism, and is measured the same way, so the comparison is rigorous.

Its integration follows the two-call protocol the library requires. The first call performs no computation, but reports how much auxiliary space is required, which is then allocated, and the second call performs the selection. Only the second is bracketed by the events that define the algorithmic scope, while the allocation and the transfers stay outside it, exactly as in the two implementations developed in this work.

The recorded data traffic and comparison count are, here too, reported as a lower bound, for the same reason as for the reference implementation on the processor (section 4.5): they include only input and output, since the library’s internal passes are not observable.

On the largest input examined, the library completes the selection in a time that stays nearly constant as k varies over four orders of magnitude, while both of this work’s implementations are burdened significantly. Only for the smallest k does map-reduce edge it out, marginally.

5.5.1 Radix selection in CUB

The library [14] uses radix selection (section 2.2.4). It never sorts the entire input; it performs a few passes, each a sequential read accompanied by a frequency count, exactly the pattern the architecture executes best. The number of passes is also determined by the width of the data type: selecting eight elements costs essentially the same as selecting one hundred and thirty thousand.

This independence from k is the fundamental difference from both of this work’s algorithms. Bitonic sort is burdened as k grows: the sorted blocks grow and cease to fit in the tile (section 5.2.1). Map-reduce is burdened even more (section 5.3.1). Radix selection has no equivalent breaking point. It is not indifferent to the data, though, in the sense of section 2.2.4: it depends on the distribution of digits in the values.

5.6 Limiting factors

On the graphics unit the limiting factors must be examined separately for each measurement scope, because they are not the same. In the end-to-end scope the bus to the host dominates (section 5.4). What follows therefore concerns the algorithmic scope, where the limits of the device itself become visible.

For small k , bitonic sort is clearly memory-bound, reaching about three-fifths of the measured bandwidth ceiling, while exploiting only about two and a half percent of the compute one. The implementation, in other words, exploits the device efficiently; it simply has to move more volume.

The reference library makes this distinction quantitative, and it is the most instructive point of the chapter. It exploits only about one-fifth of the bandwidth ceiling, clearly less than bitonic sort, and yet finishes faster. Proximity to the ceiling measures the quality of the implementation, while the total volume of traffic measures the quality of the algorithm.

Map-reduce for small k is the only point where an implementation developed in this work outperforms the library, if only marginally. It reads the input once and rejects almost everything, so it approaches the minimum possible work for the problem. Ceiling utilisation stays modest, which points to room for improvement in the organisation of accesses.

For large k the picture changes radically: map-reduce collapses to the point where utilisation of both ceilings becomes a fraction of one percent, for lack of work to perform. As explained in section 5.3.1, at these large values of k the overwhelming majority of threads fail to fill their local heap in time, leaving most of the device idle.

Bitonic sort is burdened as well, but far more mildly: utilisation of the memory ceiling drops to roughly one-third, as the loss of fusion in shared memory increases traffic without a corresponding increase in useful work. It nevertheless stays functional and, for the largest k , ends up two orders of magnitude faster than map-reduce.

Overall, neither implementation is limited by a factor that further code optimisation could lift. Bitonic sort would benefit only from a reduction in traffic, that is, a different network. Map-reduce would benefit only from a different decomposition for large k , so that parallelism would not depend on the ratio n/k . In both cases the limit is algorithmic, not implementational.

Chapter 6

NPU implementation

6.1 Design choices

The neural processing unit differs fundamentally from the two previous architectures. It is not a device on which a program runs in the usual sense, but an array of tiles configured in advance, with no cache, no dynamic memory allocation, and no ability to change its dataflow at runtime.

The design question therefore changes radically too. On the processor and the graphics unit, the question was how the algorithm is distributed across threads. Here, the question is what small, fixed unit of work the array can execute efficiently, and how the host will feed it without interruption.

6.1.1 Development flow with MLIR-AIE

No programming environment equivalent to that of the graphics unit exists. This work relies on the MLIR-AIE framework [13] and the accompanying IRON interface [20]. Development requires two separate artefacts, produced by different toolchains and combined at the end.

The first is the description of the array itself: which tiles are used, which data channels connect them, and the pattern by which data moves to and from main memory. This description is written in a domain-specific language embedded in Python, translated into an intermediate representation, and finally into a binary hardware configuration file. The second is the compute kernel executed by each tile, written in C++ using the architecture’s vector primitives and compiled separately.

The consequence that governs everything else is that this configuration is fixed at compile time. The topology, the sizes of the intermediate buffers, and the data-transfer pattern are decided before each run and do not depend on the problem’s parameters. Changing k or the input size changes nothing on the device; it only changes how the host splits the input and interprets the results.

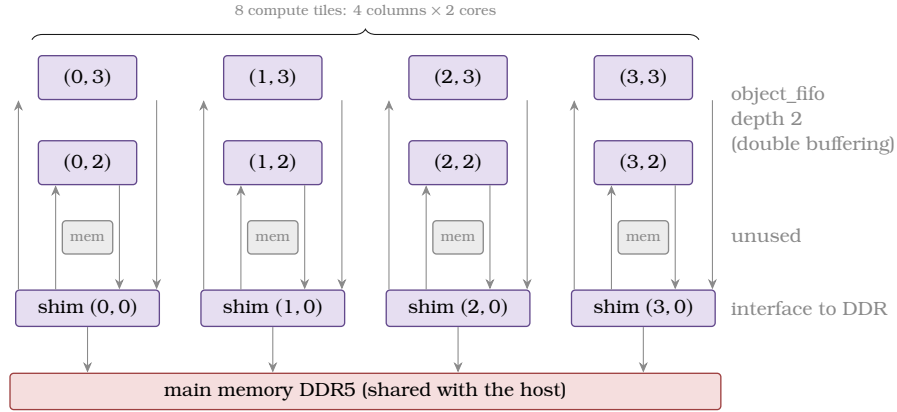
6.1.2 Tile topology and object_fifo

The array is organised into columns. Each column has one shim tile that communicates with main memory and, above it, compute tiles.

Data transfer is not programmed with explicit copy instructions, but declared as a channel between two tiles. Each channel is backed by a number of intermediate buffers, set here to two; this choice is what enables overlap: while a tile processes one buffer, the direct memory access mechanism fills the other. Without it, each tile would stall on every transfer. This technique is known as double buffering.

Each compute tile has its own pair of channels to its column's shim tile, one for input and one for output. There is no communication between compute tiles, a deliberate choice: the work assigned to the array is by definition independent per tile and requires no exchange.

The overall organisation is shown in figure 6.1. The amount of data the array processes in a single dispatch follows from the product of the above. Each tile receives chunks of 1024 elements and processes 128 such chunks per dispatch, so the batch size scales with however many tiles the algorithm in question uses. Inputs larger than this are split into successive batches.



Per core: 128 chunks $\times$ 1024 int32 elements. Per batch: $8 \times 128 \times 1024 = 1\,048\,576$ elements.

Figure 6.1: The tile topology of the implementation and the object_fifo queues that connect them. Each column has one interface tile (shim) that communicates with main memory and two compute tiles, each of which connects to its column's shim through a pair of depth-two queues: one for input and one for output. The depth of two allows the core to process one chunk while the next is transferred. The memory tiles in the middle row are unused, since each core consumes a portion of the input disjoint from the others and no sharing or redistribution of data between them is required.

6.2 Bitonic sort

An essential difference from the two previous chapters must be stated up front here, since it affects the interpretation of every measurement: the array does not execute truncated bitonic sort with the level schedule of section 4.2.

The reason is structural. The schedule requires comparisons between elements arbitrarily far apart, hence data exchange between tiles and multiple passes over the entire input. The topology of section 6.1.2 offers neither: the tiles do not communicate with one another, and every element passes through the array exactly once.

The implementation therefore reframes the problem. The array takes on a fixed, local task: it independently sorts successive sequences of sixteen elements. The host then performs the selection, merging these sorted sequences into a heap of size k .

This division of labour is not arbitrary. Because each sequence comes back sorted, the host can abandon it as soon as it encounters an element that does not exceed the heap's root: everything that follows in the same sequence is necessarily worse. The sorting the array performs is therefore not an end in itself, but a mechanism that lets the host discard sixteen elements at a time instead of one.

6.2.1 Compute tile kernel

Bitonic sort on an AI Engine tile is also the subject of an official platform example [17], which, however, performs a complete sort on a single tile.

The size sixteen follows directly from the hardware: it is exactly the number of 32-bit integers that fit in a vector register of the architecture. Sorting sixteen elements with a bitonic network requires ten compare-exchange steps, each executed entirely within the register.

Each step follows the same pattern as in section 4.1.1: a shuffle brings each element next to its partner, the minimum and maximum are computed, and a mask selects which of the two each position keeps so as to produce the correct direction. The entire network executes without any access to the tile's memory: the data is loaded once, undergoes ten register-level transformations, and is stored once.

Each kernel invocation processes a chunk of 1024 elements, i.e., sixty-four independent sequences. These sequences have no dependencies between them, which lets the compiler pipeline them and hide the latency of the individual instructions.

6.2.2 Scaling to multiple tiles

Because the sequences are independent, scaling to more tiles is trivial: it suffices to give each tile a different chunk of the batch. Neither synchronisation nor data exchange is required, and the only thing that changes is the transfer pattern declared in the device configuration.

This capability was put to use: the implementation was extended to use two compute tiles per column instead of one, reaching eight in total. The extension yielded a noticeable improvement, confirming that up to that point the implementation had indeed been compute-bound on the array.

More important than the improvement itself, however, was that the extension shifted the bottleneck elsewhere. With eight tiles the array stopped being the bottleneck,

and time is now spent almost entirely on the host. Further scaling to even more tiles would have had no effect.

6.3 Map-reduce with local heaps

The second algorithm follows the same work-distribution principle, with an even more limited role for the array. Four compute tiles are used, one per column, and the kernel does not sort: for each chunk of the input it computes a summary, namely the best value it contains.

The host maintains the heap of size k and uses the summaries as a filter. If the best value of an entire chunk does not exceed the heap's root, the chunk is rejected in its entirety, without examining any of its elements. Only for chunks that pass the filter does the host traverse the individual elements.

This filter presupposes that the heap already has a realistic threshold before the first chunk is examined. On the processor and the graphics unit this happens automatically, since the heap is bulk-initialised from the first k elements of the input; here it does not. The host instead samples, in advance, a portion of the input equal in size to the larger of k and two percent of n , and inserts it directly into the heap before the first job is dispatched to the array, so that the filter works effectively from the start.

The division of roles is thus the reverse of the intuitive one: the array does not perform the selection but produces the information that lets the host perform it cheaply. This pattern, where the accelerated part is a filter rather than the computation itself, matches the nature of the array: a fixed, local, fully predictable dataflow operation.

6.4 Memory management and data traffic

The neural processing unit has no memory of its own. Data does not cross a bus to a separate device, as with the graphics unit, but already resides in the same physical memory as the host's: what is called a transfer is in fact the synchronisation of a shared buffer.

In terms of transfer cost, this difference is indeed an advantage: there is no bandwidth gap as in section 5.4. But there is a reverse side too. Because the memory is shared, every byte the host prepares for the array consumes the same bandwidth the processor would use if it performed the computation itself. Preparation is not free; it is, as will be seen immediately, the dominant cost.

6.4.1 Pipelining and the role of the host

So that the array does not sit idle between successive batches, execution is organised as a three-stage pipeline with two alternating sets of buffers. While the array processes one batch, the host prepares the next and merges the results of the previous one.

The structure of the pipeline is given in figure 6.2. Measuring the individual stages is the most illuminating result of the entire chapter. For an input of sixteen million integers, split into sixteen batches and averaged over fifteen repetitions, time is distributed as shown in table 6.1.

Stage	Time (ms)	Percentage
Batch preparation	4.84	58 %
Result merging	2.07	25 %
Waiting for the array	0.69	8 %
Dispatching work	0.10	1 %
Initialisation	0.08	1 %
Other	0.56	7 %
Total	8.34	100 %

Table 6.1: Time breakdown of the offload region by stage. The “Other” row covers the portion of the measurement not attributed to any of the other stages, consistently present in every repetition.

The conclusion is unambiguous: less than one tenth of the time is spent waiting for the array, and ninety-two percent is work the host itself performs. A pipeline is only as fast as its slowest stage, and the slowest stage is not on the device.

This also explains why, for this architecture, the algorithmic-scope measurement must not be read as the tiles’ compute time, as already noted in section 3.3.2: the scope encloses the entire offload procedure, most of which executes on the processor.

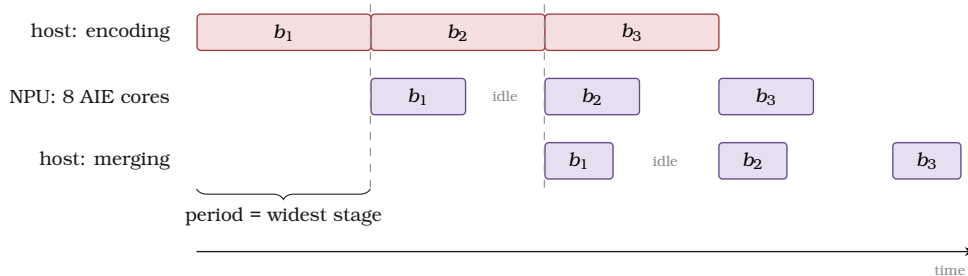


Figure 6.2: The three-stage pipelining of the offload. While the tiles process batch b_i , the host is already encoding b_{i+1} into the second buffer and merging the results of b_{i-1} . Because the three stages overlap, the total time is not their sum but the product of the number of batches and the duration of the widest stage. In this implementation that stage is host encoding: only it executes without interruption, while the other two remain idle for part of each period. No further speedup of the AIE core can therefore reduce the total time.

6.4.2 Data encoding and types

The array’s kernel operates exclusively on 32-bit integers. It does not know, and does not need to know, what the values it compares represent. Every data type used in this work must therefore be mapped to an integer key that preserves the relative order of the values, following the platform’s type-conversion rules [21].

The mapping differs by type. Signed integers are used as is. For unsigned integers, the most significant bit is flipped. For single-precision floating-point numbers, either the sign bit or the remaining bits are flipped, depending on the sign, so that the binary comparison coincides with the arithmetic one. Half precision is first widened to single precision, an exact conversion, and then follows the same path.

The critical point is that this encoding runs on the host and forms part of the preparation stage in table 6.1. The cost of the data type on this architecture is therefore a cost of the processor, not of the device. This observation has a practical consequence: half-precision encoding, initially performed in scalar fashion, proved expensive enough to noticeably burden the dominant stage, and was replaced with a vectorised path that converts eight values per instruction.

6.4.3 Final correction on the host

Double precision does not fit into a 32-bit key. The encoding keeps the 32 most significant bits of the full 64-bit key, so two different values can map to the same key. The array therefore does not return the exact result, but something weaker: a threshold.

Correctness is restored with an additional stage on the host. The worst of the keys returned by the array determines the threshold. The host then traverses the entire input again, collects the elements whose key falls within the threshold, and performs exact selection among these alone.

The cost is one additional pass over the input, which burdens double precision exclusively. The benefit is that the result is exact and passes the same correctness check as every other type, with no relaxation of the criterion. The alternative, namely accepting an approximate result for a single type, would make the comparison across types unfair.

6.5 Reference implementation

The baseline is the same standard-library function used for the processor, executed entirely on the host. The reasons for this choice were discussed in section 3.5.

It is only worth recalling the essential point: no top-k implementation exists for the array itself.

6.6 Limiting factors

The limiting factor has already been quantified in table 6.1 and leaves no room for interpretation: the implementation is bound by the host, not by the device. The remaining question is why the host's work is so expensive.

The preparation stage, which consumes three fifths of the time, comprises one full read pass over the input, the conversion of each element into an integer key, and a write of equal volume to the shared buffer: data traffic at least double the size of the input, over the same memory and at the same bandwidth the processor would use. The merging stage adds yet another read pass over the results.

This finding leads to the chapter’s central conclusion: even if the array completed its own work instantaneously, total time would drop by less than one tenth. Speed-up cannot come from the device, because the device is not where the time is spent.

The comparison with the processor sharpens the argument. The host that feeds the unit is the same processor that solves the entire problem on its own. Here it is asked to perform nearly the same amount of data traffic merely to prepare the data for someone else. The cost of preparation is comparable to the cost of the solution itself, and that is why the offload does not pay off.

This conclusion, however, concerns this particular problem, not the architecture in general. Top-k selection has minimal computational intensity: it performs one comparison per element and reuses almost nothing. A dataflow array performs well when every element that enters it undergoes many operations, so that the cost of feeding it is amortised. Here the opposite holds, and this mismatch, not any weakness of the implementation, is the underlying cause.

Chapter 7

Results and evaluation

This chapter presents the measurement results and interprets them in terms of the mechanisms described in the three preceding chapters. Unless stated otherwise, the plots concern 32-bit integers and a uniform input distribution. The effect of data type and distribution is examined separately in sections 7.6 and 7.7.

7.1 Correctness

A precondition for any timing comparison is the correctness of the result. The main sweep comprises 2700 cases, covering all three architectures, the three algorithms, the five data types, input sizes from 2^3 to 2^{24} , and the four values of k . All were verified against the ground truth of section 3.6, and all passed: a 100% pass rate for every architecture and algorithm, with no exceptions.

Every time reported in what follows therefore corresponds to an implementation that produced the correct set of k elements, and the comparisons that follow genuinely concern the same computation.

7.2 Execution time

7.2.1 Dependence on input size

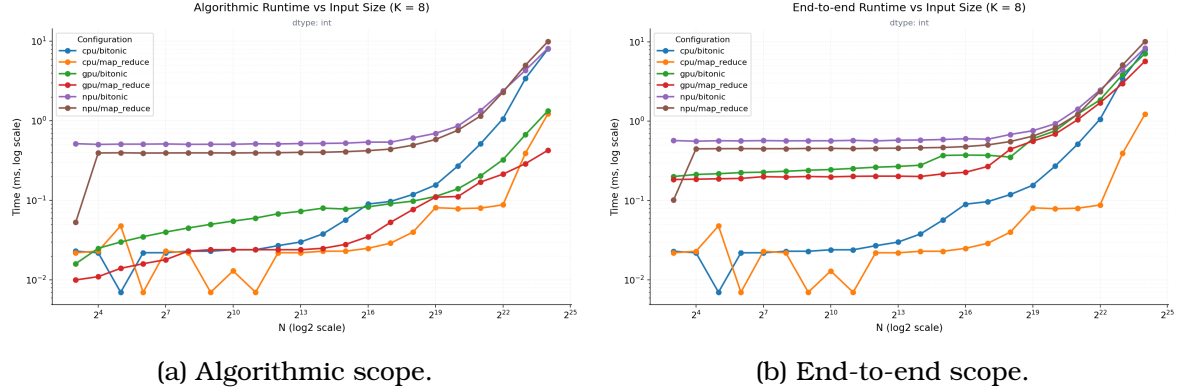


Figure 7.1: Time as a function of input size, for k equal to 8, under the two measurement scopes. The ranking of the architectures is completely overturned between the two.

In the left panel, which concerns the algorithmic scope, the graphics unit is fastest at the smallest input, completing in 0.010 ms against 0.022 ms for the processor, and again at the largest, in 0.43 ms against 1.23 ms. For nearly the entire range in between, however, the processor is the faster of the two, although the exact size at which the two swap places is itself unstable across repeated runs of the same sweep: at these small absolute times, both architectures sit close enough to the timer's resolution that the ranking there reflects measurement noise rather than a fixed architectural difference.

The neural processing unit shows a clear floor at small sizes. Its curves remain completely flat around half a millisecond up to about 2^{16} elements: throughout this range the time does not depend on input size at all, since it is dominated by the fixed cost of dispatch and waiting. The processor shows an analogous but much lower floor, on the order of five hundredths of a millisecond.

One point, though, stands out from this picture: at the smallest input size, where n and k coincide, map-reduce on the neural processing unit shows a time noticeably below the floor just described. The reason lies in the sampling described in section 6.3: when $n=k=8$, the sample that seeds the heap covers the entire input, so no work is ever dispatched to the tiles.

This picture is completely overturned in the right panel, though, once the measurement includes data transfer as well. In the end-to-end scope the processor is faster than every other architecture at *all* sizes examined, including the largest: the graphics unit kernel's 0.43 ms becomes 5.69 ms in total, while the processor stays at 1.23 ms, since for it the two scopes coincide.

The algorithmic scope shows how well each device performs the computation; the total answers whether it is worth delegating to it in the first place. The neural processing unit requires 9.97 ms at the largest input, more than any other architecture under either scope.

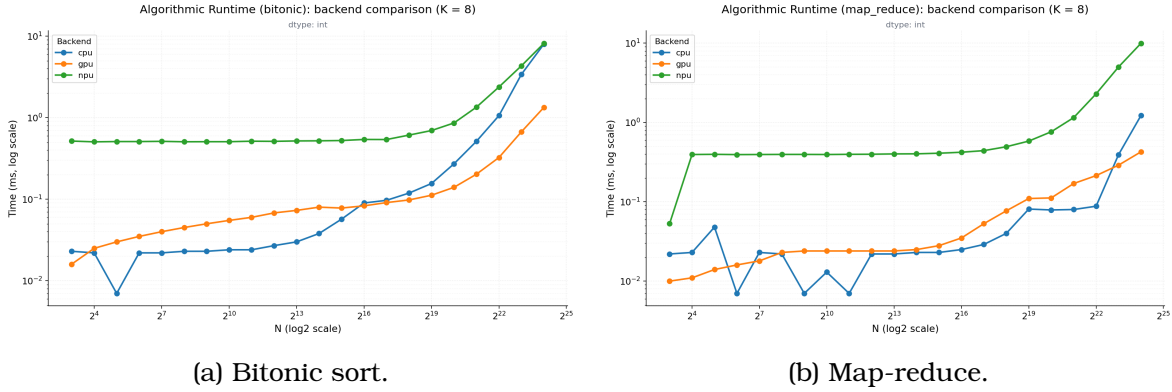


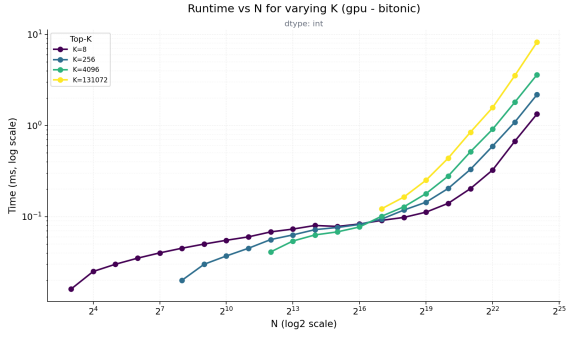
Figure 7.2: Each algorithm across the three architectures, for k equal to 8.

Comparing the same bitonic sort across the three architectures is illuminating, since the same level schedule is executed and the difference is due solely to the implementation. The two general-purpose architectures trade places along the range: the processor is faster than the graphics unit through most of the small-to-medium sizes, and the graphics unit pulls ahead only at the largest sizes, reaching a factor of six at the largest size examined, consistent with the difference in available bandwidth. As with the comparison above, the exact size at which the two swap places moves between repeated runs of the same sweep and should not be read as a fixed threshold. The neural processing unit sits close to the processor at these larger sizes, consistent with the finding that its time is spent almost entirely on the host.

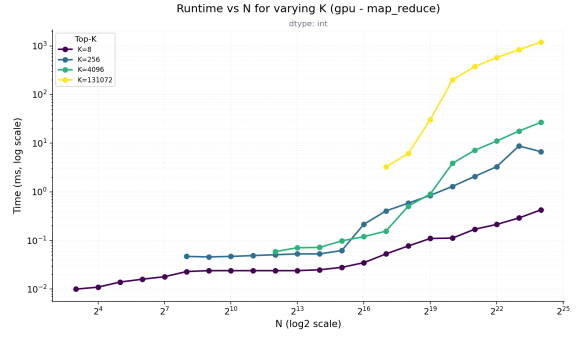
Map-reduce admits no such comparison in principle, since each architecture maintains and merges its local heaps in a fundamentally different way (chapters 4 through 6); the right panel instead shows how differently the same algorithmic idea plays out once implemented three times over. The graphics unit is the fastest of the three across almost the entire range, with the processor trailing at a roughly constant distance except at small sizes, where the same caveat about measurement noise applies as above. At the largest input the graphics unit remains the faster of the two, as above, but without the intermediate crossover bitonic sort shows. In sharp contrast to its bitonic behaviour, the neural processing unit is not close to the processor here: it stays roughly an order of magnitude slower than both architectures at every size, reaching its already-noted time at the largest input. The reason follows from section 6.6: the array's part in map-reduce is only a per-chunk filter, not a shared network schedule, so none of the structural similarity that keeps bitonic sort's curve on the neural processing unit close to the processor's carries over to this algorithm.

7.2.2 Dependence on the parameter k

The dependence on k is the point at which the ranking of the algorithms is overturned, and is perhaps this work's most significant finding.



(a) Bitonic sort, GPU.



(b) Map-reduce, GPU.

Figure 7.3: Time as a function of input size, with a separate curve for each value of k . The bitonic sort curves remain close to one another, while the map-reduce curves diverge by orders of magnitude.

The juxtaposition in figure 7.3 captures the phenomenon directly. For bitonic sort the curves of the four values of k nearly coincide, since k affects only the size of the sorted blocks. For map-reduce on the graphics unit the same curves are separated by three orders of magnitude.

Implementation	k=8		k=131072	
	ms	rank	ms	rank
CPU bitonic	8.01	4	43.33	2
CPU map-reduce	1.23	2	60.37	5
GPU bitonic	1.34	3	8.22	1
GPU map-reduce	0.43	1	1191.37	6
NPU bitonic	8.17	5	56.70	3
NPU map-reduce	9.97	6	59.69	4

Table 7.1: Algorithmic time for the two extremes of k , at an input of 2^{24} elements. The ranking concerns the six implementations of this work and does not include the reference libraries.

The fastest implementation for small k becomes the slowest for large k , a difference of three orders of magnitude. Map-reduce on the graphics unit slows down by a factor of 2800 between the two extremes, while bitonic sort on the same architecture slows down by barely a factor of six. At small k the processor's two implementations also rank higher than at large k – map-reduce overtaking bitonic sort on the graphics unit, and bitonic sort overtaking that on the neural processing unit – but the overturn itself is unaffected: the implementation cheapest at k equal to 8 is never the implementation cheapest at k equal to 131072.

This overturn is not a peculiarity of the present implementation. Independent studies on graphics units from two different manufacturers report the same dependence on k and address it with adaptive algorithm switching [3, 15]; there, the growth in time of partial-sorting methods with k is attributed explicitly to the $O(\log^2 k)$ complexity of the underlying bitonic network.

The cause was given in section 5.3.1: the heap filters only once it is full, so the number of threads that contribute is capped at the ratio n/k . For the sizes in table 7.1 this ratio is 128, i.e., the device effectively employs one hundred and twenty-eight threads out of the thousands it has available. The processor, with only eight threads, never runs into this same limit, which is why its slowdown is markedly milder.

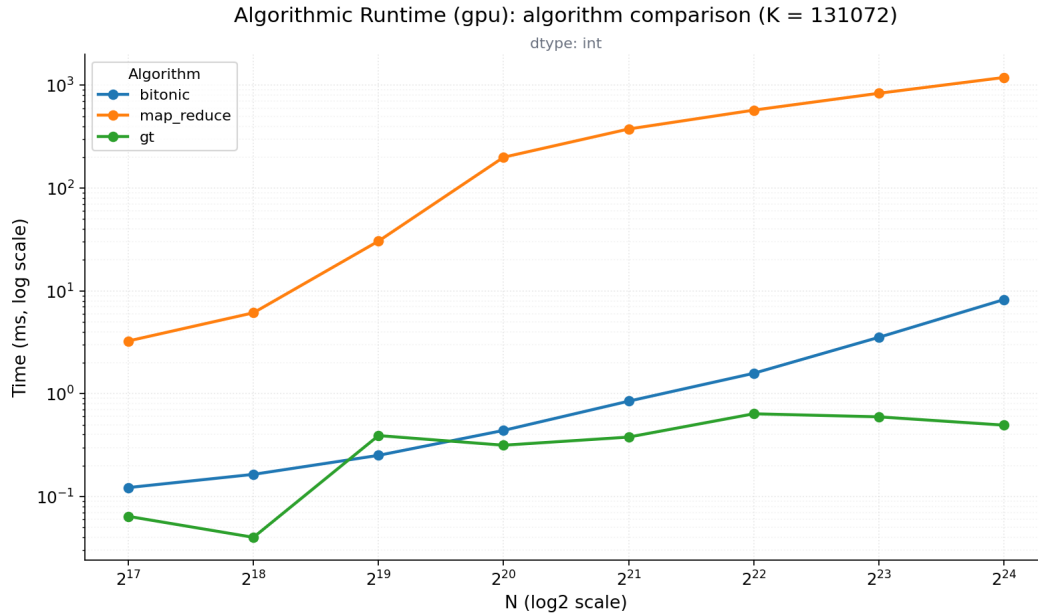


Figure 7.4: The three algorithms on the graphics unit for k equal to 131072.

The practical conclusion is that no algorithm is universally preferable. The choice between a sorting network and heap-based selection depends on the ratio n/k , not on the architecture.

7.3 Comparison across architectures

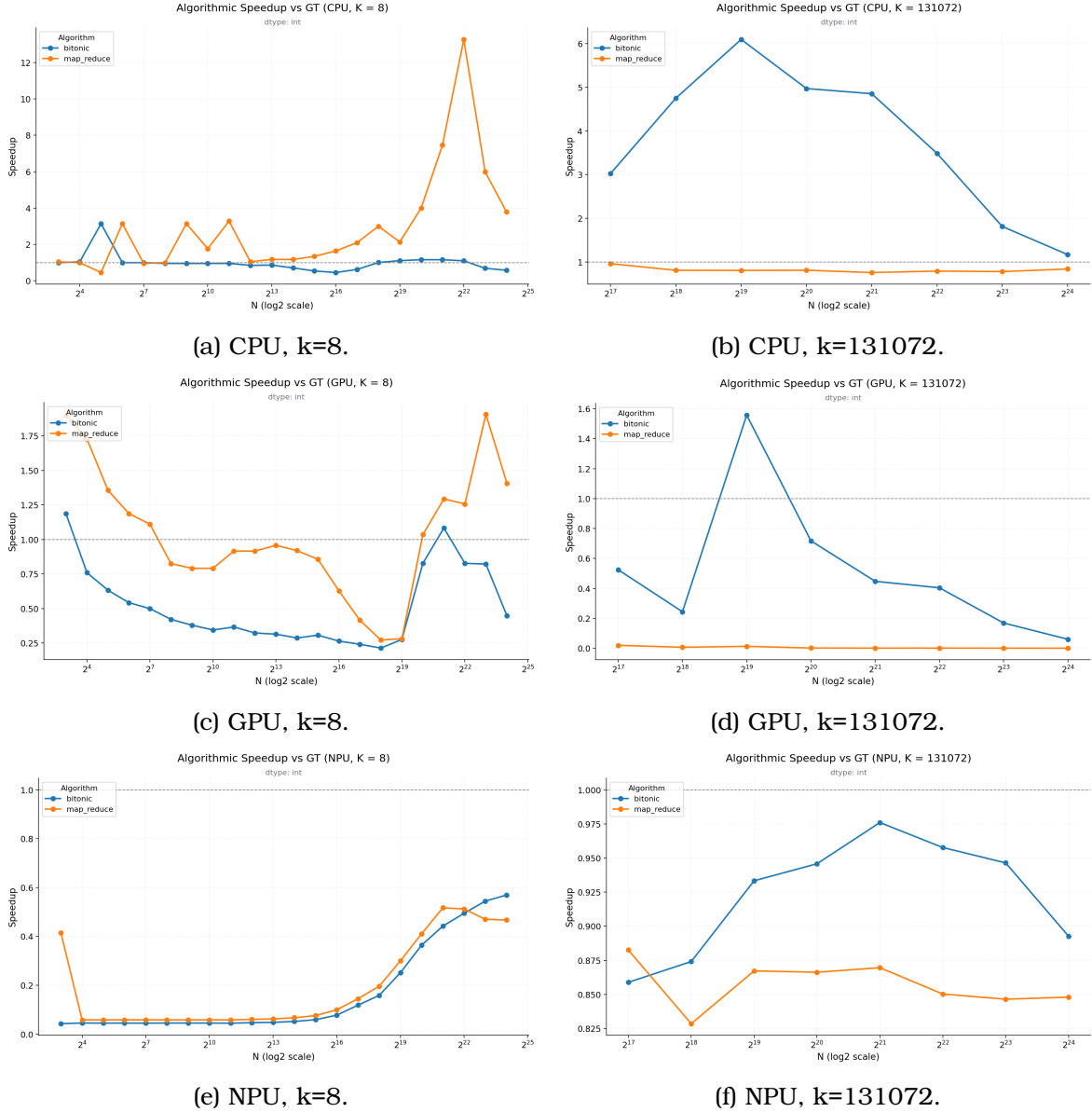


Figure 7.5: Speedup relative to the reference implementation of the same architecture. Left column k equal to 8, right column k equal to 131072, one row per architecture. Values above unity mean that this work's implementation is faster than the library.

Comparing each implementation with the reference of the same architecture gives the most honest picture, since it neutralises hardware differences.

On the processor, map-reduce is faster than the library by a factor of three for small k , although the speedup is not constant. It peaks at thirteen times for n equal to 2^{22} , where the working set (16 MB) fits exactly in the processor's L3 cache (figure 7.7), and falls back again once the input exceeds this limit. It should be remembered, though, that the limitation of section 4.5 applies here: the library is single-threaded while the implementation uses eight threads, so this factor is not purely algorithmic.

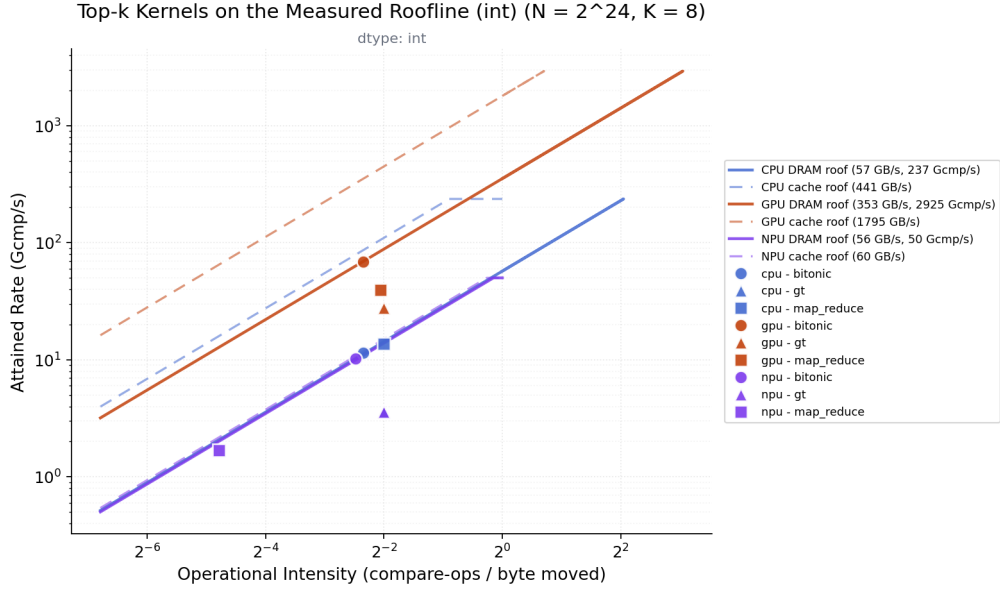
The largest speed-up in the whole figure appears elsewhere, and is a consequence of the overturn in section 7.2.2. For large k , where heap-based selection collapses, bitonic sort on the processor reaches roughly six times faster than the library. The reason is that the library, too, relies on a heap of size k and suffers the same overhead, while the sorting network stays essentially unaffected. The advantage weakens as the input grows further and the network's data traffic starts to dominate, narrowing to just above unity at the largest input.

On the graphics unit the comparison is strict, since the library runs on the same device with the same parallelism. There, map-reduce marginally outperforms for k equal to 8, while for large k the library is faster by more than a thousand times. Radix selection has no point of collapse with respect to k , as explained in section 5.5.1.

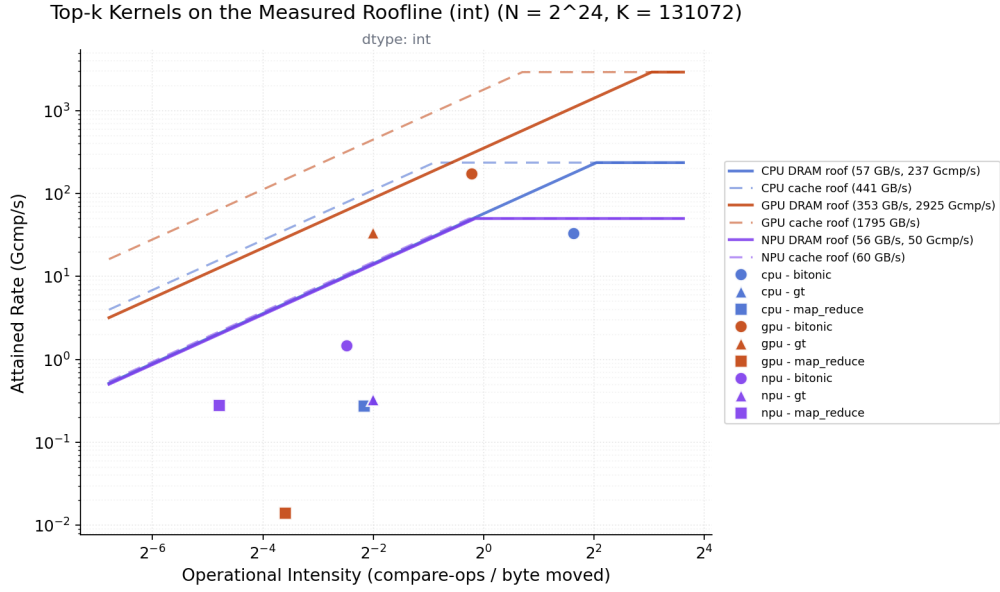
On the neural processing unit, for the integer types the figure covers, no implementation surpasses the library in any parameter region, since the very processor that would run the library is in any case consumed by preparing the data for the device (section 6.6).

The qualifier “for the integer types” is not a hedge. For floating-point types, and especially for half precision, the picture overturns and the neural processing unit does surpass the library; the explanation is given in section 7.6, after the behaviour across types is first presented.

7.4 Roofline analysis



(a) k equal to 8.



(b) k equal to 131072.

Figure 7.6: Placement of the runs against the measured ceilings, for the largest input. Restricted to the maximum size, where the working set exceeds the cache and the main-memory ceiling is the relevant one.

The diagram places each run at its level of arithmetic intensity and achieved throughput. All implementations cluster in the left-hand region, i.e., at very low arithmetic intensity, and none approaches the horizontal part of the ceiling. No implementation of this work, on any architecture, is limited by compute capability. This finding agrees with an independent measurement on data-centre accelerators, where the fastest known selection kernel utilises over ninety percent of the available bandwidth against about half of compute capability, and where time scales with each

device’s bandwidth [3].

Juxtaposing the two values of k adds two observations. In both cases bitonic sort on the graphics unit sits practically on the sloped section of its own ceiling, which visually confirms the finding of section 5.6: the implementation utilises the available bandwidth almost fully and is limited exclusively by the volume it is obliged to move.

The second observation concerns map-reduce on the same architecture. For k equal to 8 it sits near the other points, while for the large value it collapses by three orders of magnitude and moves away from both ceilings equally. This position, far from both the sloped and the horizontal section, is the signature of a limitation that is neither memory nor compute, and corresponds to the collapse of parallelism described in section 5.3.1.

This finding is structural rather than incidental. Top- k selection performs essentially one comparison per element and reuses almost nothing, so its arithmetic intensity is low by definition.

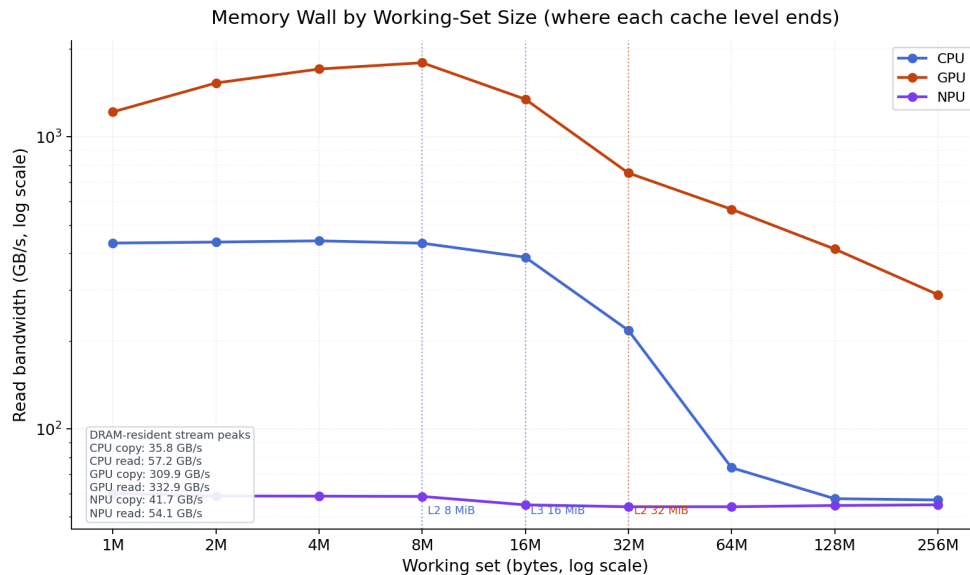
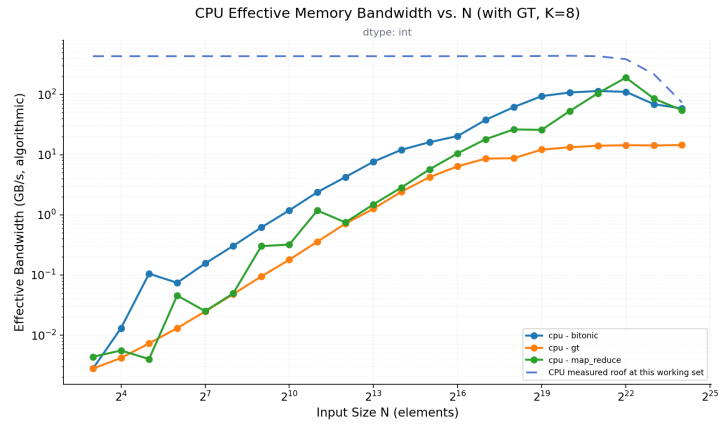


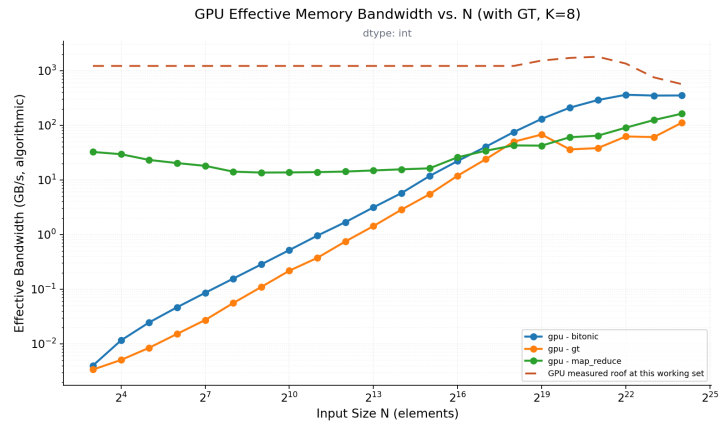
Figure 7.7: Measured read bandwidth as a function of working-set size, for the three architectures.

The ceiling against which each run is compared is not fixed but a function of its working set, per section 3.4. Figure 7.7 shows the measured ladder: the steps correspond to the levels of the memory hierarchy, and the drop toward the right-hand edge marks the point where the working set stops fitting in the last level.

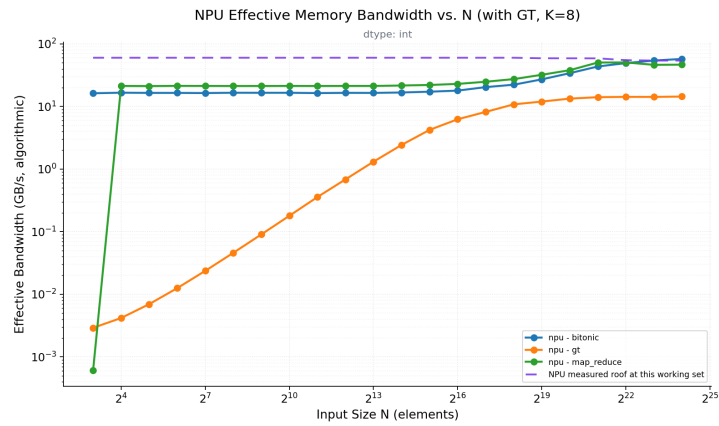
7.4.1 Memory bandwidth utilisation



(a) CPU.



(b) GPU.



(c) NPU.

Figure 7.8: Achieved bandwidth as a function of input size, one architecture per panel, with the measured ceiling drawn as a dashed line. The distance of each curve from the dashed line is the unexploited margin.

Ceiling utilisation increases with input size and levels off once the working set exceeds the cache. At the largest input, bitonic sort on the processor reaches about four-fifths of the ceiling, noticeably ahead of the graphics unit's three-fifths, while

compute-ceiling utilisation stays below five percent on both.

Juxtaposing bitonic sort with the library on the graphics unit shows that proximity to the ceiling does not equate to speed: bitonic sort clearly utilises a larger fraction of the available bandwidth than the library and is nonetheless slower. Utilisation measures the quality of the implementation; total volume measures the quality of the algorithm.

7.5 Energy assessment

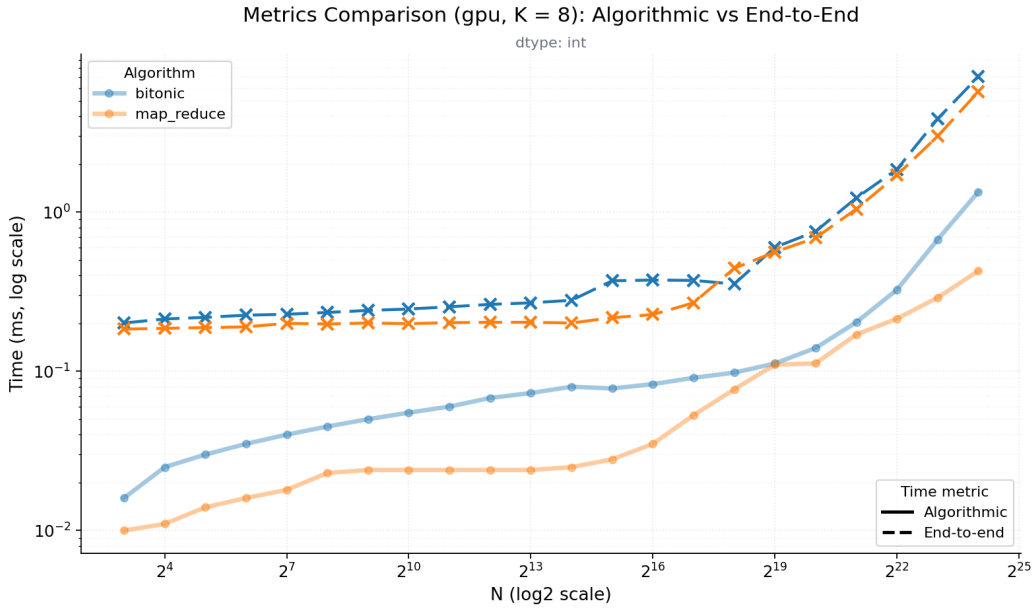


Figure 7.9: Algorithmic and end-to-end time on the graphics unit, for k equal to 8. The distance between the two curves of each algorithm is the cost of transfers and kernel launches.

Before presenting the energy measurements, figure 7.9 is useful, since it shows why the two measurement scopes diverge so sharply on the graphics unit. The end-to-end curve stays steady near three tenths of a millisecond up to about 2^{16} elements: throughout this range the kernel is essentially invisible and the time is spent entirely on transfers and launches. Even at the largest input, the end-to-end time remains a multiple of the algorithmic time, from about five times for bitonic sort to thirteen times for map-reduce.

The energy picture differs substantially from the time picture, and the difference is due to the symmetric measurement scope of section 3.3.1: each architecture is charged the total cost of solving the problem, including the host.

At the largest input and for small k , map-reduce on the processor consumes about 19 mJ per run and is the most economical implementation. The corresponding implementations on the graphics unit consume from 108 mJ to 128 mJ, i.e., up to seven times more, despite being faster in the algorithmic scope. The most expensive implementation of all is bitonic sort on the graphics unit, while the corresponding

implementation on the processor costs 118 mJ.

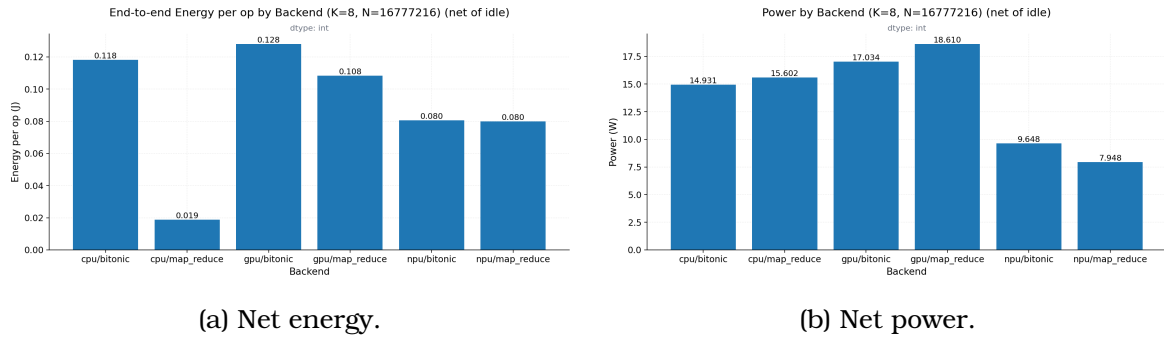


Figure 7.10: Net energy and power by architecture and algorithm, for k equal to 8.

Juxtaposing figures 7.10a and 7.10b brings out a distinction that is easy to conflate, and one that particularly concerns the neural processing unit. This device has the *lowest* power draw of all implementations, with map-reduce measuring 7.9 W against 15.6 W for the corresponding implementation on the processor and 18.6 W for the graphics unit.

Nonetheless it consumes over four times the energy of map-reduce on the processor, 0.080J against 0.019J. The reason is that energy is the product of power and time, and the device needs roughly eight times the time for the same task: the forty-nine-percent advantage in power is wiped out by the eightfold disadvantage in duration. The principle that moving data costs more energy than the computation itself does [10] holds here too; what is missing is the reuse that would amortise it. A low-power architecture pays off energetically only if it completes the task in comparable time, and for this particular problem that is not the case.

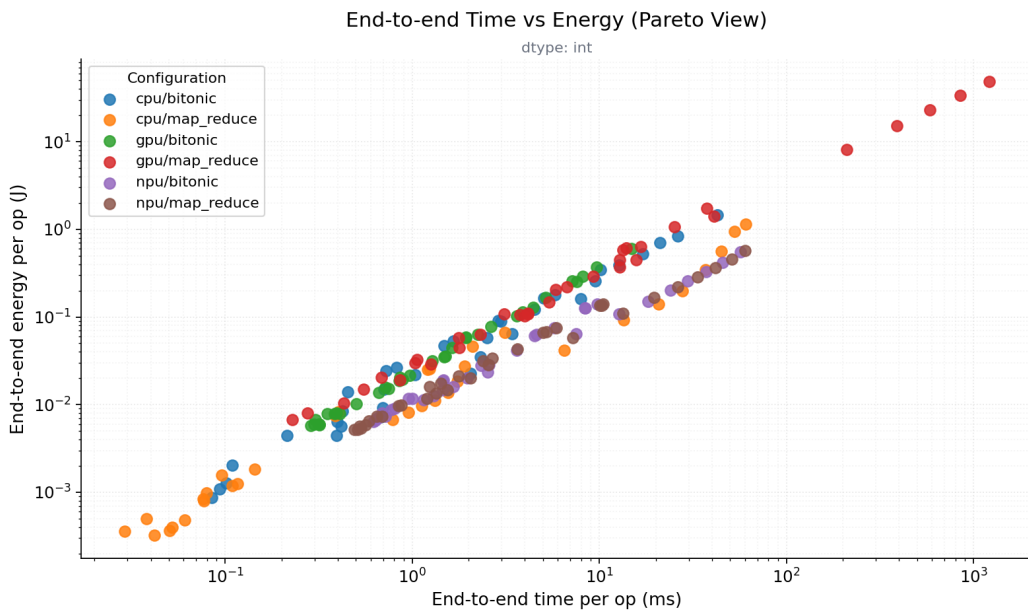


Figure 7.11: End-to-end time and total energy of each measured case. Both quantities are minimised, so the preferable cases lie toward the bottom-left corner.

Figure 7.11 places each measured case in a plane of two criteria minimised simultaneously. A case is said to be dominated by another when the latter is no worse in either quantity and is better in at least one; such a case is never worth choosing. The set of cases dominated by none other constitutes the Pareto front, i.e., exactly those choices where improving one criterion requires giving up ground on the other.

The result is illuminating precisely because it is negative. The measurements form a narrow diagonal band, with a correlation coefficient between the logarithms of time and energy of 0.97. Of the 209 cases, only two belong to the front, and both are the same implementation, map-reduce on the processor, at two neighbouring input sizes. No substantial trade-off exists between speed and energy for this problem, in other words: the fastest configuration is also the most economical.

The reason follows from the two preceding figures. Power varies among implementations by barely a factor of 2.3, while time varies by four orders of magnitude. When one factor of a product is nearly constant, the product follows the other. The possibility of trading longer duration for lower power, a genuine design lever on other systems, does not exist here.

The cases that make up the front all run on the processor. No configuration of the graphics unit or the neural processing unit is Pareto-optimal at any point of the parameter space examined.

7.6 Effect of data type

Implementation	half	int	uint	float	double
CPU bitonic	4.60	8.01	7.93	8.01	14.03
CPU map-reduce	0.39	1.23	1.22	1.22	2.35
CPU reference	13.27	4.65	4.63	5.50	5.54
GPU bitonic	0.88	1.34	1.37	1.34	2.89
NPU bitonic	7.71	8.17	8.37	8.23	25.38

Table 7.2: Algorithmic time by data type, in ms, for 2^{24} elements and k equal to 8.

This work’s implementations track the width of the type, as expected of memory-bound algorithms. Half precision is consistently the fastest, with bitonic sort on the processor completing in just over half the time of single precision, precisely because it moves half the bytes at the same rate of comparisons.

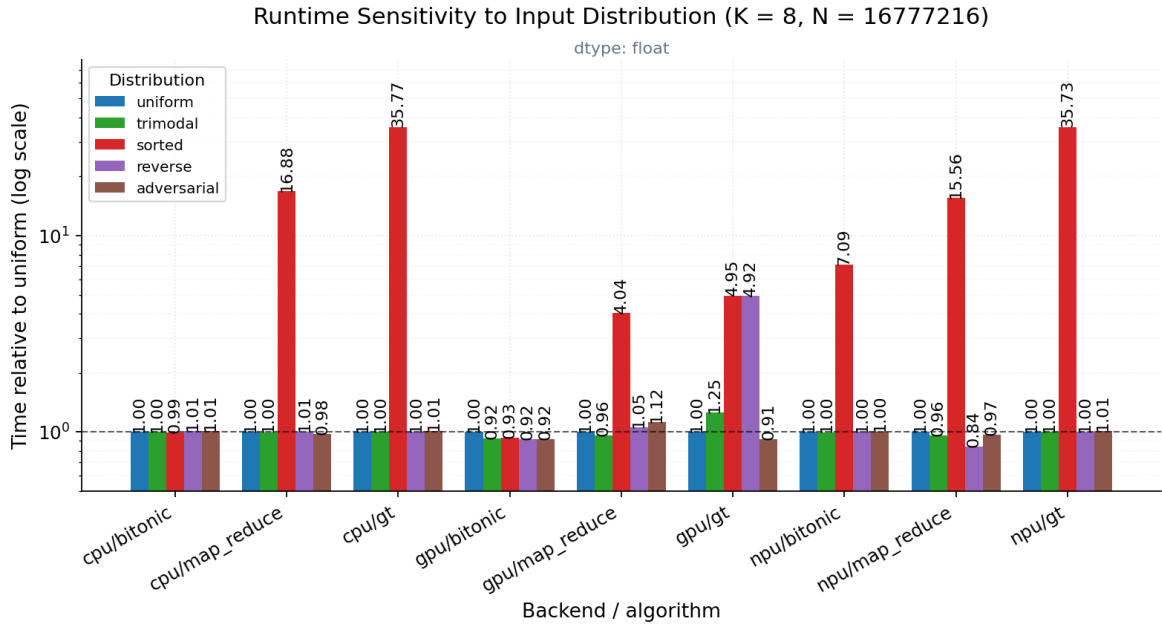
The reference implementation behaves oppositely: half precision is its slowest case. The difference reveals that the library is limited mainly by the cost of the scalar comparison (which for half precision includes a conversion), rather than by memory. The ranking among types is therefore a characteristic of the implementation, not an inherent property of the hardware.

Double precision on the neural processing unit stands out, at three times the time of any other type. The overhead is due entirely to the additional refinement pass of section 6.4.3, required because the 32-bit key is not enough to express 64-bit values.

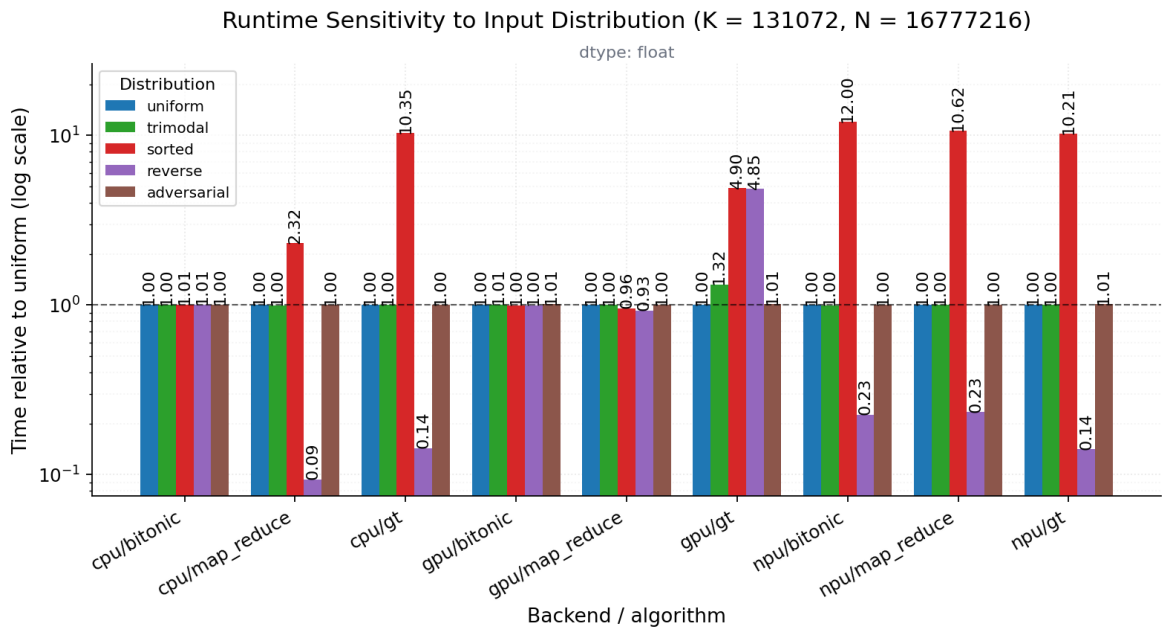
The combination of these two opposite behaviours produces the single point in the entire work where the neural processing unit outperforms its own library. For half precision, bitonic sort on the device is faster than the library in twenty-nine of the measured cases, with the largest gap reaching a factor of two and a half. The phenomenon appears more weakly in the other two floating-point types, and never appears for the integer types.

The cause is not that the device handles half precision well, but that the library handles it poorly. The array's kernel operates on 32-bit integers regardless of type, so the cost of the type shifts to host encoding, which for half precision is vectorised per section 6.4.2. The library, by contrast, performs scalar comparisons with an intermediate conversion. Offloading proves advantageous exactly where the established solution is weakest, in other words, not where the device is strongest.

7.7 Sensitivity to the input distribution



(a) k=8.



(b) k=131072.

Figure 7.12: Sensitivity of time to the input distribution, for single precision and 2^{24} elements. Each bar gives the time of one distribution relative to the time of the uniform distribution for the same implementation; unity means complete independence from the data.

Normalising against the uniform distribution lets all nine combinations be compared on the same figure, even though their absolute times differ by orders of magnitude. An implementation unaffected by the data appears as five equal bars of height one, while a sensitive one appears with bars diverging in both directions.

The figure first confirms the expected: bitonic sort on the processor and the graphics unit is practically unaffected by the distribution at both values of k , with overall variation below a factor of 1.09. This property makes it a reliable witness: if its time were to change, the change would be attributable to the measurement, not the algorithm.

Heap-based implementations show the opposite extreme, and indeed regardless of who wrote the code. The reference library varies by a factor of seventy-two between ascending and descending input, but so does our own map-reduce on the processor, by a factor of twenty-five, through the same mechanism described in section 4.3. Sensitivity to the data is therefore a property of the algorithm rather than of the implementation: whoever chooses a heap inherits it.

The graphics unit's reference library shows a third pattern: it slows down by nearly the same factor in both directions, 4.95 for sorted against 4.92 for reverse, rather than favouring one the way the heaps do. Since sorting does not alter the value distribution (section 3.7.2), this is not the radix-adversarial digit clustering of section 2.2.4 but a positional effect: the candidates surviving the critical bucket end up clustered at one end of the array or the other, which burdens the library's parallel compaction step symmetrically regardless of which end they land on.

Two further points stand out here. First, bitonic sort on the neural processing unit *is* sensitive to the distribution, unlike the other two architectures, with a variation factor of fifty-three. This sensitivity does not originate in the array, which executes a fixed network, but in the heap-based merge performed by the host, which, as shown in section 6.4.1, dominates the time.

Second, juxtaposing the two values of k isolates the cause of the sensitivity. Map-reduce on the graphics unit is sensitive for k equal to 8, where the sorted input slows it down by a factor of 4.0, but becomes nearly unaffected for k equal to 131072, where the same input changes it by only 4%. The same code, with the same data, changes behaviour because the limiting factor changes: in the second case it is not rejection that limits it but the lack of parallelism described in section 5.3.1. When the limiting factor is not the data, their distribution stops mattering.

Chapter 8

Conclusions and future work

8.1 Conclusions

This work implemented two top- k selection algorithms on three architectures and measured them over the same parameter space, with a common harness and common correctness checking. The conclusions that follow do not repeat the measurements of chapter 7, but rather what generalises from them beyond the specific hardware.

Top- k selection is a data-traffic problem. For small k , every combination of algorithm and architecture is limited by bandwidth rather than by the devices’ computational capability (section 7.4). Performance depends solely on reducing the volume of data moved, not on speeding up the computation. Bitonic sort on the graphics unit confirms this clearly: although it makes use of a larger fraction of its bandwidth ceiling than the reference library, it still loses, because it is forced to move several times as much data.

The appropriate algorithm is determined by the ratio n/k , not by the architecture. The fastest implementation for k equal to 8 is the slowest for k equal to 131072 regardless of platform (section 7.2.2), while the sorting network’s performance stays far more stable. The cause is structural rather than implementation-specific: the heap only begins to filter once it is full, so the available parallelism is capped by the ratio n/k . Similar behaviour has also been observed in the literature [15], underscoring the need for algorithms that adapt to the input data.

The measurement boundary determines which architecture “wins”. In the algorithmic scope the graphics unit is faster only at the two extremes of the size range; in the end-to-end scope the processor is faster across the entire range, including those same extremes. The ranking, that is, overturns exactly where the graphics unit appeared to win, and the only difference between the two answers is where the measurement boundary is placed. This finding has methodological significance beyond the specific problem: any speed-up claim for offloading to an accelerator that reports only kernel time is not verifiable, because it conceals precisely the cost that determines whether offloading is worthwhile.

Power and energy. The neural processing unit has the lowest power draw of any implementation and at the same time the highest energy consumption, since duration dominates the product (section 7.5).

For top-k selection with small values of k , offloading the computation to an accelerator proves disadvantageous in both energy and time compared to running map-reduce on the processor. Architectures such as NPUs (section 2.3.3) presuppose high local reuse of data to amortise the cost of moving it. Top-k selection offers no such reuse, so most of the time ends up spent preparing the data, leaving the device idle (section 6.4.1).

The sole exception is bitonic sort on the neural processing unit for half-precision numbers, where it outperforms the corresponding reference implementation. The advantage is attributable to the processor library’s poor optimisation for this particular data type (section 7.6).

8.2 Limitations

The most significant asymmetry concerns the processor’s baseline. `std::partial_sort` is strictly single-threaded, whereas this work’s implementations exploit parallelism, so every ratio of times between them conflates two causes, the difference in algorithm and the difference in thread count, as already noted in section 4.5. This choice is justified by the question this work asks, which concerns comparison with what the programmer already has available; the reader should nonetheless bear it in mind.

The measured data traffic of the reference implementations is a lower bound rather than an actual measurement, since only the input and output are counted while the libraries’ internal passes are not observable. Every ceiling-utilisation percentage reported for them in section 7.4.1 is consequently an underestimate.

For the neural processing unit there is no on-device baseline, because no top-k selection library exists for that architecture. The reference implementation runs on the host, so the relevant comparisons answer the question “is offloading worthwhile” rather than “how good is the implementation relative to the best possible one on the device”. Correspondingly, the phase-breakdown analysis in section 6.4.1 comes from separate runs with debug mode enabled rather than the standard sweep, so its percentages rest on a smaller sample of repetitions than the rest of the measurements in appendix B.

Finally, the experimental space was limited to a single system and specific ranges of values. The qualitative trends – in particular the dependence on the ratio n/k and the role of the measurement boundary – are expected to generalise to comparable systems; the exact numerical factors, however, remain closely tied to this particular test platform.

8.3 Future work

The clearest direction follows directly from the limiting factor identified for the neural processing unit. Since most of the time is spent preparing and sending data per

call, a workload that runs many top- k queries at once would amortise exactly this cost; batched selection, moreover, arises naturally in the applications mentioned in section 2.1, where selection repeats at every step or for every query.

In the same direction, reducing the host’s involvement in pipelining – with the device accessing data directly instead of explicit copying per chunk – would attack the same cost from the other side. This possibility depends on the tools and interfaces the platform exposes.

A second direction is adaptive algorithm selection. Given that the overturn in section 7.2.2 is systematic and predictable from the ratio n/k , an implementation that dynamically chooses between the heap and the sorting network would cover the entire parameter range without a point of collapse.

A third direction is extending the parameter space toward points this work touched on but did not exhaust: values of k larger than those examined, where selection approaches sorting, and inputs that do not fit in a single device’s memory, where managing the stream becomes part of the algorithm rather than of the infrastructure.

Finally, the measurement harness developed here is independent of the specific problem. The separation of the two measurement scopes, the recording of energy with common semantics across three architectures, and the placement of kernels against a measured ceiling are tools that can be applied to any workload being considered for offloading.

Appendix A

Acronyms

AIE	AI Engine
API	Application Programming Interface
AVX	Advanced Vector Extensions
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
D2H	Device to Host
DRAM	Dynamic Random Access Memory
EDP	Energy Delay Product
FIFO	First In First Out
GPU	Graphics Processing Unit
H2D	Host to Device
MLIR	Multi-Level Intermediate Representation
NPU	Neural Processing Unit
NVML	NVIDIA Management Library
RAPL	Running Average Power Limit
SIMD	Single Instruction Multiple Data
SIMT	Single Instruction Multiple Threads
SM	Streaming Multiprocessor
TOPS	Tera Operations Per Second
XRT	Xilinx Runtime

Appendix B

Reproducing the Measurements

This appendix describes the steps required to reproduce the thesis’s measurements, from installing the tools to generating the plots. The commands are given for the system described in Section 3.1 and are executed from the root of the repository, available at <https://github.com/ioannisam/topk>.

B.1 Software Requirements

All three implementations require a C++ compiler that supports the C++20 standard, together with CMake version 3.18 or newer. Vector instructions are not a compilation requirement: the choice between AVX-512, AVX2 and scalar execution is made at run time, based on the capabilities reported by the processor. The thesis’s measurements were obtained on a processor with AVX-512, so on a machine without it the CPU follows a different code path and its results are not directly comparable. Beyond these, each architecture has its own requirements.

The GPU requires the CUDA toolkit with the nvcc compiler and the CUB library, which is distributed together with it. Power measurements additionally require the NVML library, which ships with the driver.

The NPU requires the XRT runtime stack together with the XDNA device plugin, and, for compiling the kernels, MLIR-AIE within a Python environment. This environment must reside at the root of the repository under the name `npu_venv`, since both kernel compilation and the automated execution of the measurements look for it there. The user must belong to the groups that have access to the device, and the locked-memory limit must be sufficient; otherwise the device is not visible to the runtime stack. The `XILINX_XRT` variable must point to the installation location of XRT.

Energy measurement reads the RAPL counters through `/sys/class/powercap`, which requires elevated privileges. In a single run, without these the measurement is explicitly reported as unavailable rather than reporting zero values, as described in Section 3.3.3; the corresponding sweep, by contrast, does not even start and terminates with an error, so that no empty dataset is produced. Finally, the analysis infrastructure requires Python with the libraries declared in the repository; the corresponding environment is created automatically the first time it is run.

B.2 Compilation

Compiling all implementations is done with a single command:

```
make build-all
```

Each architecture can also be compiled separately, using the `build-cpu`, `build-gpu` and `build-npu` targets, which is useful on a system that does not have all three units. The resulting executables are placed in the `build` directory, two per architecture: one for the top-k measurements and one for the roofline measurements of Section 3.4.

Compilation for the NPU additionally produces the hardware configuration files. Because their absence leads to the corresponding measurements being silently skipped, their presence is checked explicitly:

```
make verify-artifacts
```

The check covers the two configuration files and the three top-k measurement executables, so it presupposes a full compilation of all three implementations; after compiling only a single architecture it fails by default.

B.3 Running the Experiments

The recommended way to reproduce the results is not to run the individual steps, but a single command that executes the entire procedure:

```
make benchmark GPU_W=50
```

The reason this is the preferred route is not convenience, but consistency. The order of the steps matters: if the system conditions are pinned after the measurements have started, or if a sweep is run with executables from an earlier compilation, the results are not comparable to one another. The command enforces the correct order and leaves no room for a step to be skipped.

The sequence that runs consists of six stages. First, the previous results and old executables are deleted, so that nothing survives from an earlier run. Next, all three implementations are compiled and the presence of the resulting files is checked, using the check from Section B.2. In the third stage, the system conditions are pinned and read access to the energy counters is granted. In the fourth, the four sweeps are run in sequence; in the fifth, the plots for each of them are generated; and in the sixth, the original system conditions are restored.

Two preconditions are checked before anything else: the availability of `sudo` and the existence of the `npu_venv` environment from Section B.1. If either is missing, the command terminates immediately, without altering the system or the existing results.

Three properties of the command are worth noting. Administrator privileges are requested only once, at the beginning, and remain active for the entire duration, so the run requires no further intervention afterwards. The conditions are restored in every case, even if the run is interrupted by the user or terminates with an error, so that the system is not left pinned. Finally, the failure of one sweep does not cancel

the rest: it is logged as a warning and the procedure continues, so that a partial run nonetheless produces whatever data it was possible to measure.

The GPU_W argument sets the power limit of the GPU, which, according to Section 3.8, must remain constant throughout the sweep. It should also be stressed that the command deletes the existing results as its first step, so any previous measurements that need to be kept must first be copied elsewhere.

B.3.1 Running Individual Steps

For a partial or repeated run, each stage is also available separately. The system conditions are controlled with:

```
make pin ARGS=50          # governor, boost off, GPU cap 50 W
make pin-show             # report the current state
make unpin                # restore governor, boost, GPU cap
```

The measurements are divided into four independent sweeps, each of which writes its raw results to a separate file:

```
make measure-cases        # timing over the parameter space
make measure-energy        # energy, requires RAPL access
make measure-dists        # input-distribution sensitivity
make measure-roofline     # bandwidth, compare-exchange roofs
```

Each target accepts additional arguments through the ARGS variable, which are forwarded to the sweep driver and allow the parameter space to be restricted to a subset of it. This is useful for quickly checking the installation, where a full sweep would be excessively time-consuming. When the stages are run separately, responsibility for the correct order and for the stability of the conditions between sweeps rests with the user.

B.3.2 Execution Parameters

Table B.1 gives the full set of parameters accepted by each executable, their permitted values, their defaults, and the values with which the thesis's sweeps were run.

The defaults are those of the executable and in several cases do not match the values used in the sweeps. The value range extends up to 10^9 instead of the default, and the reason is substantive: with a range of only a few thousand values and an input of millions of elements, value repetitions would be so dense that the problem would turn into a choice among ties, which would artificially favour algorithms that reject quickly.

This range applies to four of the five types. For half, the executable clamps the bounds to ± 65504 , the largest finite value it can represent, so sampling actually takes place over $[0, 65504]$. For this type, ties are not avoided, both because of the narrower range and because above 2048 half does not represent every integer. This is an inherent property of the type rather than a methodological choice, but it must be taken into account when interpreting its results.

The seed is not fixed, but is generated deterministically from the q and k of each case, so that two different cases never receive the same input while each case remains reproducible.

The values in the fourth column refer to the main sweep. The two special sweeps restrict certain parameters: the distribution sweep examines q from 20 to 24, only the float and half types, and the k extremes 8 and 131072, with two seeds per case; the energy sweep examines q from 16 to 24 with three repetitions and without correctness checking, so that the check is not counted toward the consumption.

Parameter	Values	Default	In the sweeps	Meaning
q	integer 0-63	—	3-24	Input size $n = 2^q$; the only mandatory parameter
k	positive integer	n	8, 256, 4096, 131072	Number of requested elements; capped at n
mode	min max	max	max	Selects the smallest or largest elements
dtype	int uint float double half	int	all five	Element type
algo	bitonic map_reduce gt	bitonic	all three	Algorithm to run
run	full trunc both	trunc	trunc	Variant of the bitonic network
seed	non-negative integer	42	generated from q, k	Seed of the random-number generator
dist	uniform normal trimodal sorted reverse adversarial	uniform	uniform; five in the distribution sweep	Input distribution
min / max	integers	0 / 1000	0 / 10^9	Value range; requires $\max \geq \min$; clamped to the range of the type
threads	positive integer	0 (automatic)	8	Number of threads; ignored by the NPU
verify	true false	false	true	Checks correctness against the ground truth
debug	true false	true	false	Verbose diagnostic output

Table B.1: The execution parameters and their defaults.

B.4 Generating the Plots

The analysis infrastructure reads the raw results of the sweeps, converts them into tables in CSV format, and generates the plots:

```
make plot ARGS="--plot all"
```

The `-plot` argument selects which plots will be produced, while `-input` specifies which sweep will be read; the default is the timing sweep. The energy, distribution and roofline plots read their own files, so it is usually preferable to use the combined targets, which run the measurement and the corresponding plot generation together:

```
make benchmark-cases
make benchmark-energy
make benchmark-dists
make benchmark-roofline
```

The resulting files are organised by data type and by measurement category, together with the tables from which they were derived. Plots that concern the machine itself, such as the bandwidth scale of Section 3.4, are placed in a separate directory, since they do not depend on the data type.

Two options affect the interpretation of the plots and are worth mentioning. `-energy-metric` selects between gross energy, net energy, or both together, which is also the executable's default; the thesis's sweeps explicitly request the net figure, for the reasons given in Section 3.3.3. `-error-bars` selects the kind of dispersion band drawn around the values, among standard deviation, deciles, or no band at all, with standard deviation as the default. A band is drawn only where there are at least two samples per point, namely in the distribution sweep, which uses two seeds per case, and in the energy sweep, which repeats each case three times. In the automated run, the bands are disabled for all four sweeps, so the plots it produces do not include them.

Bibliography

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [2] N. Sismanis, N. Pitsianis, and X. Sun, “Parallel search of k-nearest neighbors with synchronous operations,” in *2012 IEEE Conference on High Performance Extreme Computing (HPEC)*, Waltham, MA, USA, 2012, pp. 1–6.
- [3] J. Zhang, A. Naruse, X. Li, and Y. Wang, “Parallel top-K algorithms on GPU: A comprehensive study and new methods,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’23)*, Denver, CO, USA, 2023.
- [4] K. E. Batcher, “Sorting networks and their applications,” in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1968, pp. 307–314.
- [5] A. Shanbhag, H. Pirk, and S. Madden, “Efficient top-K query processing on massively parallel hardware,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, Houston, TX, USA, 2018, pp. 1557–1570.
- [6] D. A. Palmer and M. Florea, “Neural processing unit,” U.S. Patent US8655815B2, assigned to the University of California, San Diego, 2014, application US13/011,727, filed 21 January 2011; granted 18 February 2014.
- [7] H. Esmailzadeh, A. Sampson, L. Ceze, and D. Burger, “Neural acceleration for general-purpose approximate programs,” in *45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2012, pp. 449–460.
- [8] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 269–284.
- [9] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *44th International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [10] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [11] AMD, “Versal adaptive SoC: The industry’s first adaptive compute acceleration platform,” AMD, White Paper, 2021, <https://www.amd.com/>

content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/
versal/adaptive-soc-paper.pdf.

- [12] A. Rico, S. Pareek, J. Cabezas, J. Noguera *et al.*, “AMD XDNA NPU in Ryzen AI processors,” *IEEE Micro*, vol. 44, no. 6, 2024.
- [13] AMD/Xilinx, “MLIR-AIE: An MLIR-based toolchain for AMD AI engine-enabled devices,” <https://github.com/Xilinx/mlir-aie>, 2024.
- [14] NVIDIA, “CCCL: CUDA core compute libraries,” <https://github.com/NVIDIA/cccl>, 2024.
- [15] C. Lin, M. Yang, C. Wang, and C. Huang, “Adaptive top-K selection: Eliminating performance cliffs across all K values on AMD GPUs,” AMD ROCm Blogs, Feb. 2026, <https://rocm.blogs.amd.com/software-tools-optimization/adaptive-topk/README.html>.
- [16] Y. Samaga, V. Yerram, S. R. Babbula, P. Jain, and P. Netrapalli, “A faster generalized two-stage approximate top-K,” *Transactions on Machine Learning Research*, 2026, <https://openreview.net/forum?id=izqZ1Crpjz>.
- [17] AMD, “Bitonic SIMD sorting on AI engine for float datatypes,” Vitis Tutorials: AI Engine Development, 2024.1, 2024, <https://docs.amd.com/r/2024.1-English/Vitis-Tutorials-AI-Engine-Development/Bitonic-SIMD-Sorting-on-AI-Engine-for-float-Datatypes>.
- [18] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [19] Intel, “Intel intrinsics guide,” <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>, 2024.
- [20] AMD, “IRON: A close-to-metal programming framework for AMD NPUs,” <https://github.com/amd/IRON>, 2024.
- [21] —, “AI engine kernel and graph programming guide (UG1079): Casting and datatype conversion,” 2024, <https://docs.amd.com/r/en-US/ug1079-ai-engine-kernel-coding/>.